

第二次作业 — 系统架构选择及分工说明

一、系统架构选择及说明

1.1 架构风格选择

本系统采用**分层架构（Layered Architecture）风格**，结合**事件驱动**模式进行设计。选择理由如下：

1.1.1 分层架构说明

系统从逻辑上划分为以下四层：

层级	名称	职责	对应组件
L1	表示层 (Presentation)	用户交互界面展示，接收用户输入，展示系统响应	用户客户端、管理员客户端
L2	应用层 (Application)	处理用例流程，协调领域对象，管理事务边界，驱动定时采集任务	系统主控（事件分发器）、ScheduledTaskService（定时任务服务）
L3	领域层 (Domain)	核心业务逻辑：调度算法、计费规则、队列管理、故障处理策略	DispatchService、BillingService、QueueService、MonitorService
L4	基础设施层 (Infrastructure)	数据持久化、消息推送、外部接口	数据库、消息中间件

1.1.2 选择分层架构的理由

- 职责分离清晰**：表示层只负责界面逻辑，领域层封装核心业务规则，适合团队分工协作
- 业务逻辑可测试**：领域层可独立于UI和数据层进行单元测试，核心调度和计费算法可充分验证
- 易于扩展**：计费规则变更、调度策略增加（如可选的最短时长策略）只需修改领域层，不影响其他层
- 符合课程要求**：分层架构是UML建模中最常用的架构风格，便于绘制系统顺序图和类图

1.1.3 通信协议选择：RESTful API

客户端与服务器端之间采用 **RESTful API** 风格进行通信，理由如下：

- 语义清晰**：每个系统事件（指令）直接映射为一个 REST 接口，URL 体现资源，HTTP 方法体现操作
- 无状态通信**：每个请求独立，服务器不需维护客户端会话状态，便于水平扩展
- 广泛支持**：各类客户端（小程序/APP/Web）均可通过 HTTP 调用，开发调试方便
- 与分层架构匹配**：表示层通过 HTTP 调用应用层接口，应用层再将请求转发至领域层处理

指令与 REST 接口映射表：

系统指令	HTTP方法	REST路径	说明
createNewAccount	POST	/api/accounts	创建新账号

系统指令	HTTP方法	REST路径	说明
set_pwd	PUT	/api/accounts/{car_Id}/password	设置密码
login	POST	/api/auth/login	登录认证
E_chargingRequest	POST	/api/charging/requests	提交充电申请
Modify_Amount	PUT	/api/charging/requests/{car_Id}/amount	修改充电量
Modify_Mode	PUT	/api/charging/requests/{car_Id}/mode	修改充电模式
Query_Car_State	GET	/api/charging/requests/{car_Id}/state	查询车辆队列状态
Start_Charging	POST	/api/charging/sessions	开始充电
Query_Charging_State	GET	/api/charging/sessions/{car_Id}	查询充电状态
End_Charging	DELETE	/api/charging/sessions/{car_Id}	结束充电
Request_Bill	GET	/api/bills?carId=&date=	查看账单
Request_DetailedList	GET	/api/bills/{Bill_Id}/details	查看详单
powerOn	POST	/api/piles/{pile_Id}/power/on	启动充电桩
setParameters	PUT	/api/piles/{pile_Id}/parameters	设置参数
Start_ChargingPile	POST	/api/piles/{pile_Id}/run	运行充电桩
powerOff	POST	/api/piles/{pile_Id}/power/off	关闭充电桩
Query_PileState	GET	/api/piles/{pile_Id}/state	查询充电桩状态
Query_QueueState	GET	/api/queues/state	查询队列状态

1.1.4 实时状态更新机制：客户端定时轮询

充电桩状态的更新采用**客户端定时轮询（Polling）**机制，理由如下：

- 1. **符合作业要求**：作业明确要求"定时刷新所有充电桩的状态并在客户端显示"，轮询是最直接的实现方式
- 2. **实现简单可靠**：客户端定时调用 GET /api/piles/{pile_Id}/state 拉取最新状态，无需维护长连接
- 3. **与 REST 风格一致**：复用已有的 REST 接口，不需引入 WebSocket 等额外协议

轮询流程：

- 管理员客户端启动后，设置轮询间隔（默认 N 秒）
- 每隔 N 秒，客户端向服务端发送 Query_PileState(pile_Id) 请求
- 服务端 MonitorService 实时读取充电桩状态数据，返回
(workingState, TotalChargeNum, TotalChargeTime, TotalCapacity)
- 客户端对比新旧数据，仅更新变更部分，减少界面重绘开销

1.1.5 服务器端事件驱动补充

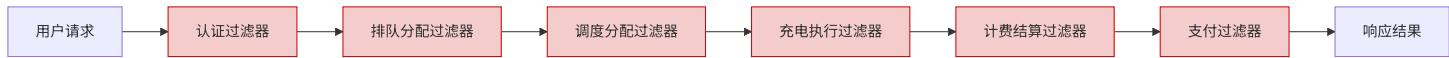
系统在以下服务器内部场景中采用事件驱动模式：

- **充电桩故障检测**：监控系统检测到故障后发布故障事件，触发调度服务执行对应策略（优先级/时间顺序/充电中故障恢复）
- **队列状态变更**：车辆入队/出队/换队时发布事件，通知相关模块更新内部状态

1.2 废弃架构方案及废弃原因

以下列出在设计过程中被考虑但最终废弃的三种架构方案，并说明废弃原因。

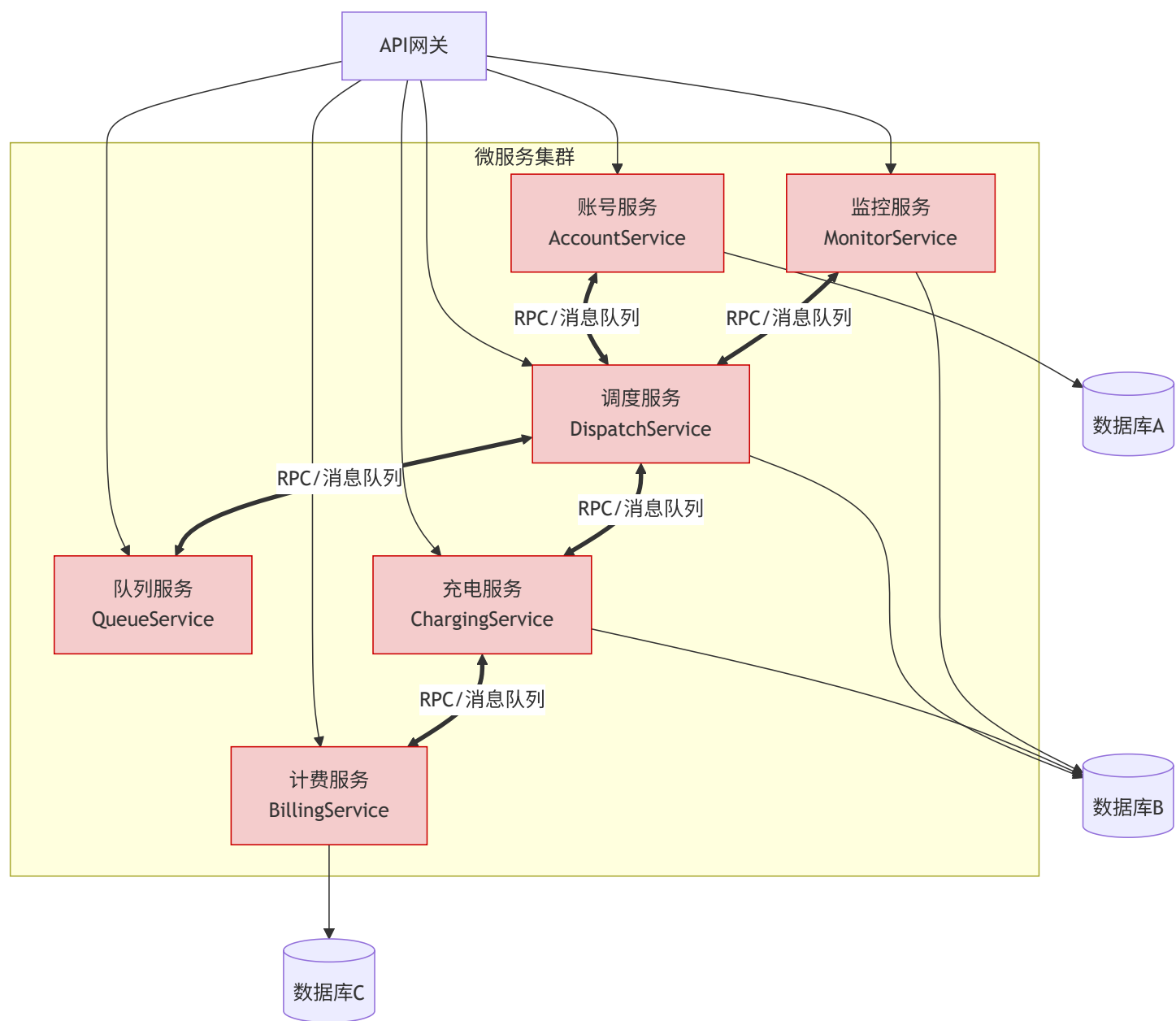
1.2.1 废弃方案一：管道-过滤器架构（Pipe-Filter）



废弃原因：

1. **业务流程非线性**：充电业务存在大量分支和回路——用户在排队区可更换队列（回退到排队分配）、充电中可修改协议电量（不需要经过排队分配）、故障恢复需要跳跃到调度分配。管道模型要求数据严格单向流动，无法优雅处理这些场景。
2. **状态管理困难**：每个过滤器是无状态的，但充电请求的生命周期状态（排队中→等待中→充电中→已完成）需要跨多个过滤器维护，管道模型缺少统一的状态管理中心。
3. **交互复杂性**：用户可随时取消充电、修改请求，这要求管道内的某个过滤器能"打断"上游的流水线，管道模型对此支持不足。

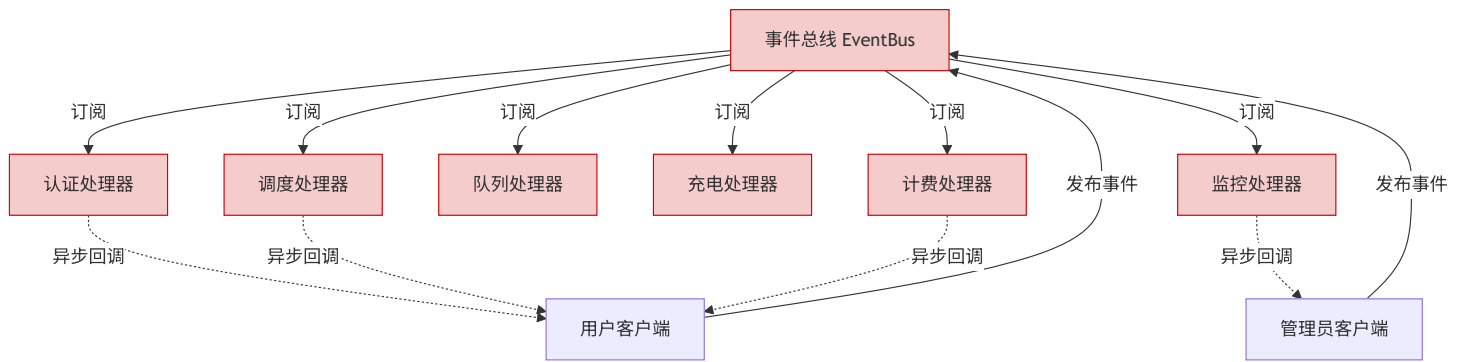
1.2.2 废弃方案二：微服务架构 (Microservices)



废弃原因：

- 1. **规模不匹配**：本系统服务于单个充电站（2快充+3慢充充电桩），并发量有限。微服务架构的运维开销（服务发现、负载均衡、配置中心）远超这个规模的实际需要。
- 2. **分布式事务复杂化**：充电请求从提交到完成涉及调度→队列→充电→计费多个步骤，微服务间需分布式事务协调，而单体+分层可通过本地事务保证一致性。
- 3. **开发与调试成本高**：5人小组需同时理解和调试多个独立服务，增加了协作复杂度，不利于课程作业的交付。
- 4. **部署复杂度**：微服务需要容器化部署（Docker/K8s），而分层架构单体应用可直接部署，更符合课程验收要求。

1.2.3 废弃方案三：纯事件驱动架构（Pure Event-Driven）



废弃原因：

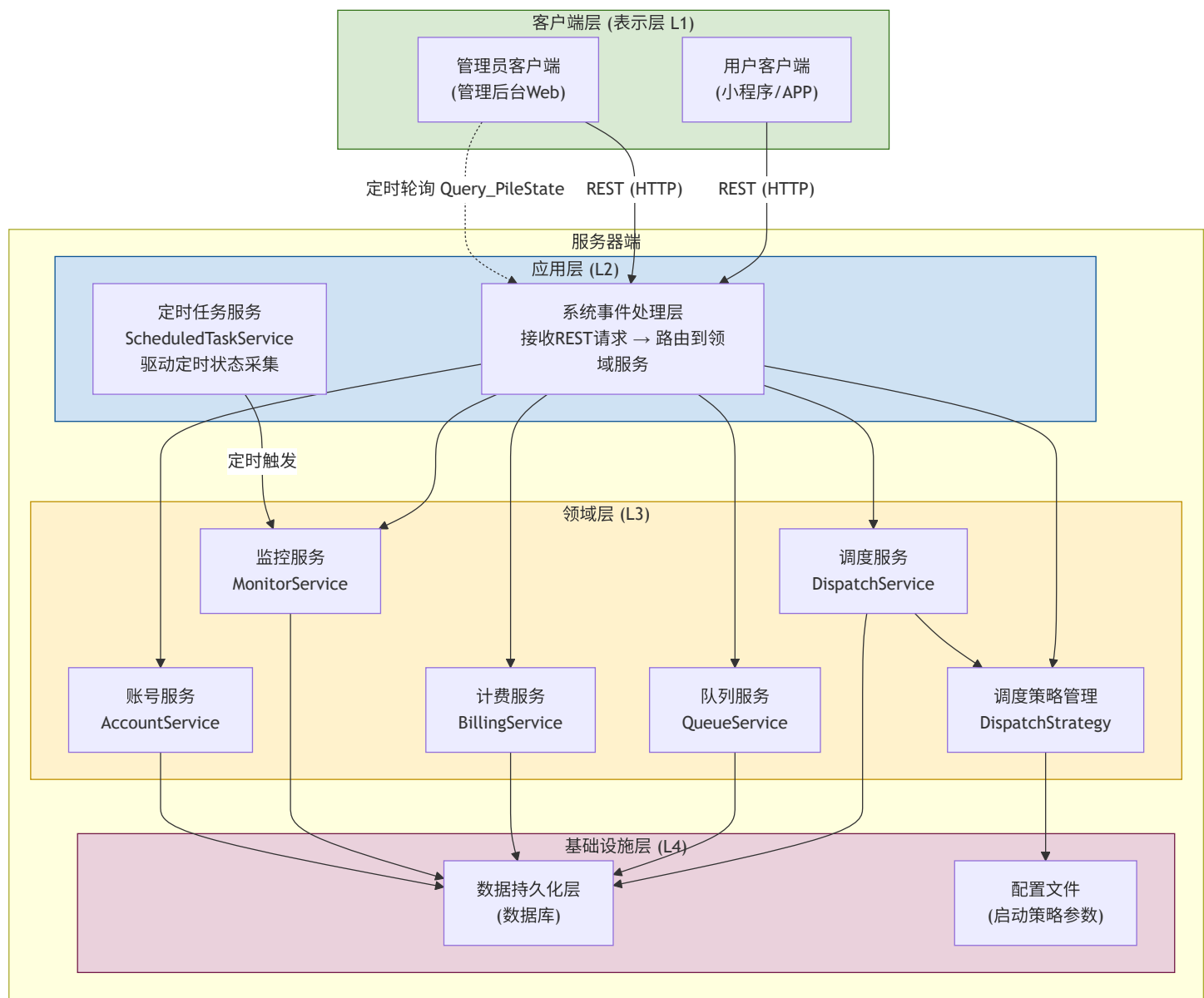
- 1. **同步操作占主导**：用户登录、提交充电申请、查看状态、结束充电等绝大多数操作为同步请求-响应模式，用户需要立即得到反馈。纯事件驱动适合异步处理，但同步场景需要额外的回调或轮询机制，增加不必要的复杂度。
- 2. **调试困难**：事件在总线上异步流转，没有清晰的调用链，出现问题难以追踪根因。
- 3. **时序依赖风险**：充电服务流程有严格时序（先排队→再等待→再充电→再计费），纯事件驱动下缺乏流程编排机制，容易出现事件乱序。

决策：保留事件驱动作为**服务器端内部补充模式**，用于故障检测和队列状态变更等异步场景，但整体架构不以此为主。

1.2.4 方案对比总结

维度	管道-过滤器	微服务	纯事件驱动	分层架构（选定）
业务适配度	差（非线性流程）	中	差（同步操作为主）	优
开发复杂度	低	高	高	低
可测试性	中	中	差（异步测试困难）	优
运维成本	低	高	中	低
扩展性	差	优	优	中
团队适配度（5人）	中	差	差	优

1.3 系统部署拓扑（逻辑视图）



图例说明：

- 实线箭头 (→)：RESTful API 同步调用，客户端发起请求，服务端返回响应
- 虚线箭头 (-.->)：定时轮询请求，客户端按固定间隔发起
- 绿色 (客户端层)：表示层，负责用户交互
- 蓝色 (应用层)：请求路由与协调
- 黄色 (领域层)：核心业务逻辑
- 紫色 (基础设施层)：数据持久化

层间通信规则：

- 表示层仅能调用应用层接口（通过 REST API）
- 应用层调用领域层服务处理业务逻辑，驱动定时任务
- 领域层通过基础设施层进行数据持久化
- 禁止跨层调用：表示层不能直接访问领域层或基础设施层
- 禁止反向依赖：领域层和基础设施层不依赖应用层

在这个拓扑中：

- **用户客户端**通过 REST 调用登录、充电申请、查看账单等接口
- **管理员客户端**通过 REST 调用运行充电桩、设置参数等接口，同时通过定时轮询刷新充电桩状态
- **应用层路由**接收 REST 请求，根据指令类型分发到对应的领域服务
- **应用层定时任务服务(ScheduledTaskService)定期触发领域层 MonitorService 采集充电桩状态——这是合法的 L2→L3 调用**，遵循层次单向依赖
- **领域服务之间**通过直接依赖协作（如调度服务通过 DispatchStrategy 获取当前激活策略后执行调度任务）
- **配置文件**存储启动参数指定的默认调度策略，系统启动时由基础设施层读取后注入到 DispatchStrategy

1.4 架构与用例的映射

用例	涉及层级	通信方式	核心领域服务
注册（UC-22）	表示层→应用层→领域层 →基础设施层	REST (POST/PUT)	AccountService
登录（UC-01）	表示层→应用层→领域层 →基础设施层	REST (POST)	AccountService
充电申请（UC-02）	表示层→应用层→领域层 →基础设施层	REST (GET/POST/PUT/DELETE)	DispatchService, QueueService
查看账单（UC-12）	表示层→应用层→领域层 →基础设施层	REST (GET)	BillingService
查看详单（UC-12a）	表示层→应用层→领域层 →基础设施层	REST (GET)	BillingService
运行充电桩（UC-23）	表示层→应用层→领域层 →基础设施层	REST (POST/PUT)	—（直接管理ChargingPile）
查看充电桩状态 （UC-14）	表示层→应用层→领域层	REST (GET) + 定时轮询	MonitorService
查看队列状态（UC-15）	表示层→应用层→领域层	REST (GET)	QueueService
管理调度策略（UC-41）	表示层→应用层→领域层 →基础设施层	REST (GET/PUT) + 配置文件	DispatchStrategy, ConfigFile

二、静态结构设计

2.1 领域对象识别

2.1.1 实体（Entity）

实体	说明
ChargingStation充电桩	充电站聚合根，持有充电区（含多个充电桩），每个充电桩各自管理其排队区与等待区
User用户 、 Vehicle车辆	用户与车辆
ChargingRequest充电请求	一次充电请求，贯穿排队/等待/充电状态
ChargingPile充电桩	充电桩（快充/慢充）
PileQueue桩队列	每个充电桩对应的排队结构（容量约束）
ChargingSession充电会话	一次实际充电会话
BillingRecord账单记录 、 DetailedBill详单记录 、 PaymentOrder支付订单	账单与支付
FaultEvent故障事件	故障事件
OperationReport运营报表	运营报表

2.1.2 值对象（Value Object）

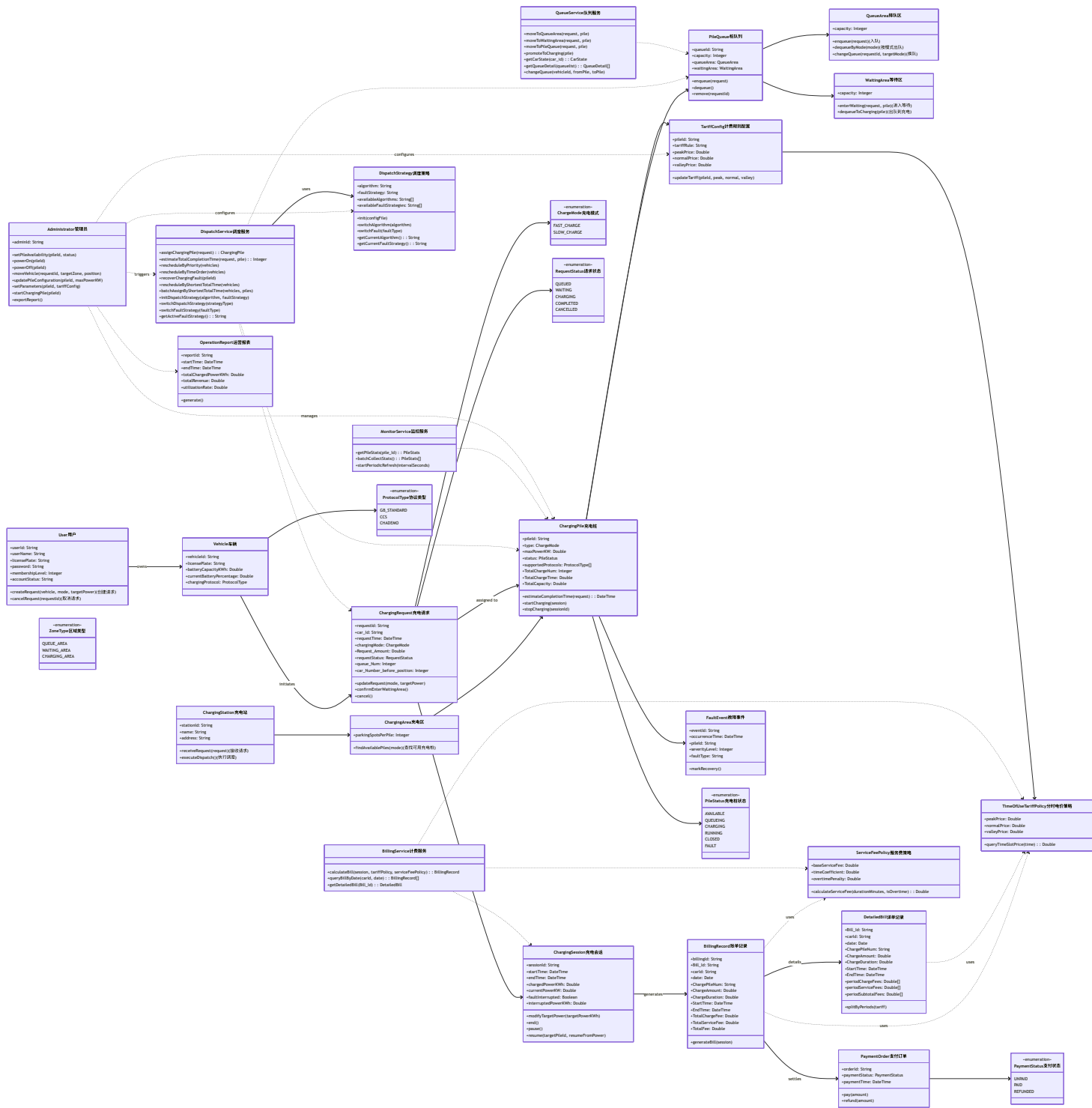
值对象	说明
TimeOfUseTariffPolicy分时电价策略	分时电价策略值对象
ServiceFeePolicy服务费策略	服务费策略值对象
TariffConfig计费规则配置	计费规则配置值对象
ChargeMode充电模式 、 PileStatus充电桩状态 、 RequestStatus请求状态	枚举类型
ProtocolType协议类型 、 ZoneType区域类型 、 PaymentStatus支付状态	枚举类型

2.1.3 领域服务（Domain Service）

领域服务	职责
DispatchService调度服务	按当前激活策略进行分配与再调度
QueueService队列服务	车辆在排队区/等待区/充电区的流转控制
BillingService计费服务	统一费用计算与账单生成
MonitorService监控服务	充电桩状态定时采集与推送
DispatchStrategy调度策略	封装策略选择与切换逻辑

2.2 领域模型类图

本类图仅使用**定向关联**(-->)、**依赖**(..>)和**继承**(<|--)三种关系。



2.3 关键约束与业务规则映射

业务规则	实现方式
快充/慢充分流	ChargingRequest充电请求.chargingMode + DispatchService调度服务.assignChargingPile

业务规则	实现方式
三区队列（每桩独立）	每个充电桩的 PileQueue桩队列 管理自己的 QueueArea排队区 + WaitingArea等待区
桩队列容量	PileQueue桩队列.capacity （默认可设为 4）
最短完成时间目标	DispatchService调度服务.estimateTotalCompletionTime
请求变更与取消	ChargingRequest充电请求.updateRequest/cancel
故障再调度（五种策略）	FaultEvent故障事件 + DispatchService调度服务 的五个重调度方法
调度策略可切换	DispatchStrategy调度策略 — 系统启动参数决定默认值，管理员运行时可切换
分时电价	TimeOfUseTariffPolicy分时电价策略.queryTimeSlotPrice
服务费（基础费 + 时长/超时）	ServiceFeePolicy服务费策略.calculateServiceFee
计费规则配置	TariffConfig计费规则配置 存储充电桩级别的计费规则和三个时段电价
定时状态刷新	MonitorService监控服务.startPeriodicRefresh

2.4 用例级静态结构类图

按第二次作业要求，使用依赖、定向关联、继承三种关系，围绕6组用例绘制静态结构类图。

2.4.1 注册与登录用例类图

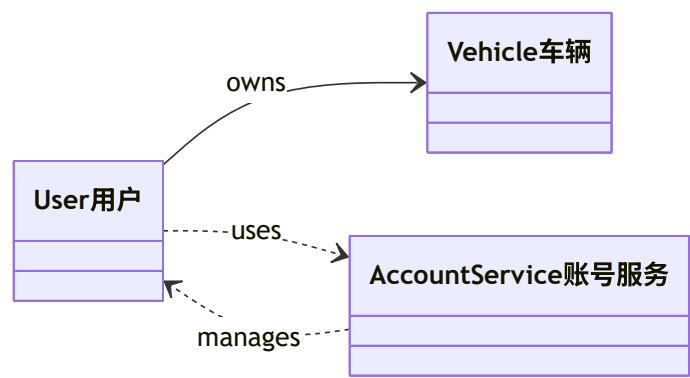


表 2.1 注册与登录用例类说明

类名	属性	方法
User用户	userId, userName, licensePlate, password, membershipLevel, accountStatus	createNewAccount(car_Id, userName, car_Capacity), set_pwd(password), login(car_Id, password)
Vehicle车辆	vehicleId, licensePlate, batteryCapacityKWh, currentBatteryPercentage, chargingProtocol	—
AccountService账号服务	—	validateAccount(car_Id), createAccount(user), setPassword(userId, password),

类名	属性	方法
		authenticate(car_id, password)

2.4.2 充电申请用例类图

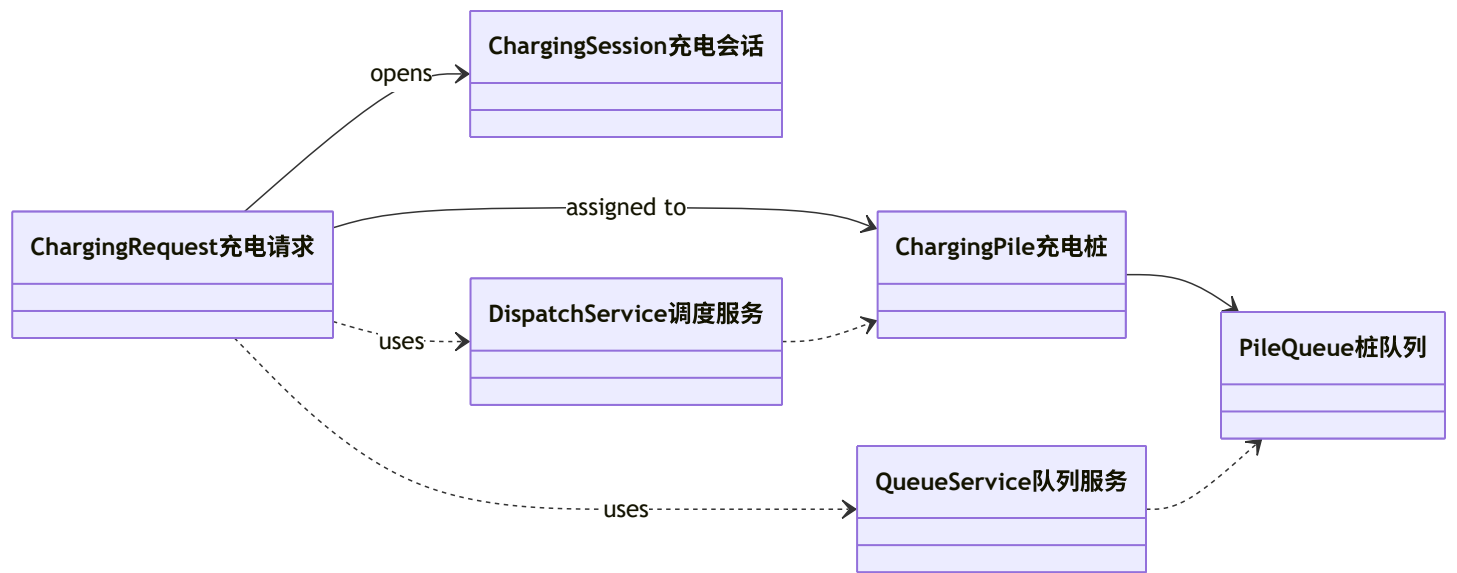


表 2.2 充电申请用例类说明

类名	属性	方法
ChargingRequest充电请求	requestId, car_id, requestTime, chargingMode, Request_Amount, requestStatus, queue_Num, car_Number_before_position	updateRequest(mode, targetPower), cancel()
ChargingPile充电桩	pileId, type, maxPowerKW, status, supportedProtocols, TotalChargeNum, TotalChargeTime, TotalCapacity	estimateCompletionTime(request), startCharging(session), stopCharging(sessionId)
ChargingSession充电会话	sessionId, startTime, endTime, chargedPowerKWh, currentPowerKW, faultInterrupted, interruptedPowerKWh	modifyTargetPower(targetPowerKWh), end(), pause(), resume(targetPileId, resumeFromPower)
DispatchService调度服务	—	assignChargingPile(request), estimateTotalCompletionTime(request, pile)
QueueService队列服务	—	enqueue(request, pileId), getCarState(car_id), dequeue(pileId)
PileQueue桩队列	queueId, capacity	enqueue(request), dequeue(), remove(requestId)

2.4.3 查看账单与详单用例类图

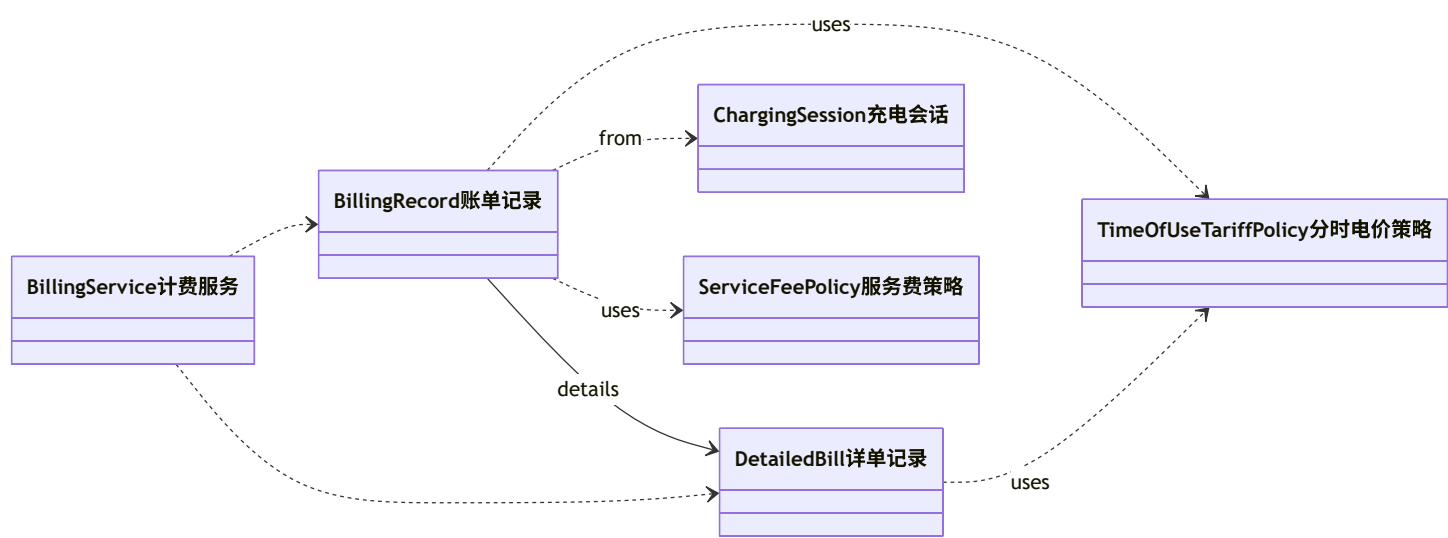


表 2.3 查看账单与详单用例类说明

类名	属性	方法
BillingRecord账单记录	billingId, Bill_Id, carId, date, ChargePileNum, ChargeAmount, ChargeDuration, StartTime, EndTime, TotalChargeFee, TotalServiceFee, TotalFee	generateBill(session)
DetailedBill详单记录	Bill_Id, carId, date, ChargePileNum, ChargeAmount, ChargeDuration, StartTime, EndTime, periodChargeFees[], periodServiceFees[], periodSubtotalFees[]	splitByPeriods(tariff)
BillingService计费服务	—	calculateBill(session), queryBillByDate(carId, date), getDetailedBill(Bill_Id)
TimeOfUseTariffPolicy分时电价策略	peakPrice, normalPrice, valleyPrice	queryTimeSlotPrice(time)
ServiceFeePolicy服务费策略	baseServiceFee, timeCoefficient, overtimePenalty	calculateServiceFee(durationMinutes, isOvertime)

2.4.4 运行充电桩用例类图

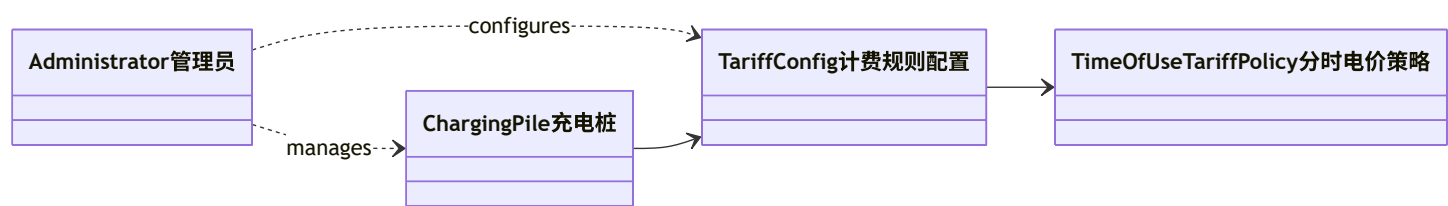


表 2.4 运行充电桩用例类说明

类名	属性	方法
Administrator管理员	adminId	powerOn(pile_Id), powerOff(pile_Id), setParameters(pile_Id, tariffConfig, peakPrice, normalPrice, valleyPrice), Start_ChargingPile(pile_Id)
ChargingPile充电桩	pileId, type, maxPowerKW, status, supportedProtocols	—
TariffConfig计费规则配置	pileId, tariffRule, peakPrice, normalPrice, valleyPrice	updateTariff(pileId, peak, normal, valley)
TimeOfUseTariffPolicy分时电价策略	peakPrice, normalPrice, valleyPrice	queryTimeSlotPrice(time)

2.4.5 查看充电桩状态与队列状态用例类图

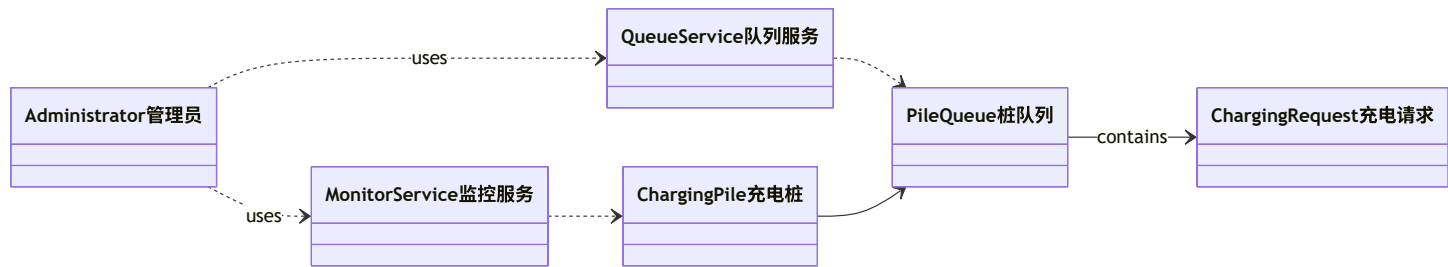


表 2.5 查看充电桩状态与队列状态用例类说明

类名	属性	方法
MonitorService监控服务	refreshInterval	getPileStats(pile_Id), batchCollectStats(), startPeriodicRefresh(intervalSeconds)
QueueService队列服务	—	getQueueDetail(queueId)
ChargingPile充电桩	pileId, type, status, TotalChargeNum, TotalChargeTime, TotalCapacity	—
PileQueue桩队列	queueId, capacity	getVehicles(), calculateWaitTime(vehicle)

类名	属性	方法
ChargingRequest充电请求	car_Id, requestTime, Request_Amount	—
Administrator管理员	adminId	Query_PileState(pile_Id), Query_QueueState(queueId)

2.4.6 管理调度策略用例类图

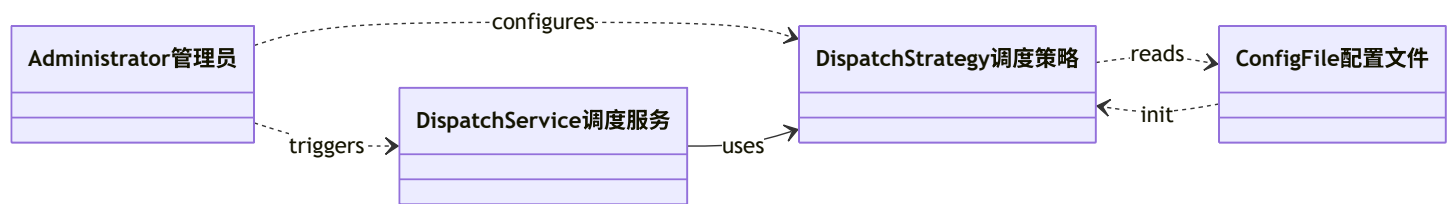


表 2.6 管理调度策略用例类说明

类名	属性	方法
DispatchStrategy调度策略	algorithm, faultStrategy, availableAlgorithms[], availableFaultStrategies[]	init(configFile), switchAlgorithm(algorithm), switchFault(faultType), getCurrentAlgorithm(), getCurrentFaultStrategy()
DispatchService调度服务	—	assignChargingPile(request), rescheduleByPriority(vehicles), rescheduleByTimeOrder(vehicles), recoverChargingFault(pileId), rescheduleByShortestTotalTime(vehicles), batchAssignByShortestTotalTime(vehicles, piles), initDispatchStrategy(algorithm, faultStrategy), switchDispatchStrategy(strategyType), switchFaultStrategy(faultType)
ConfigFile配置文件	algorithm, faultStrategy	—
Administrator管理员	adminId	setDispatchStrategy(strategyType), setFaultStrategy(faultType)

2.5 UML活动图（客户充电服务业务流程）

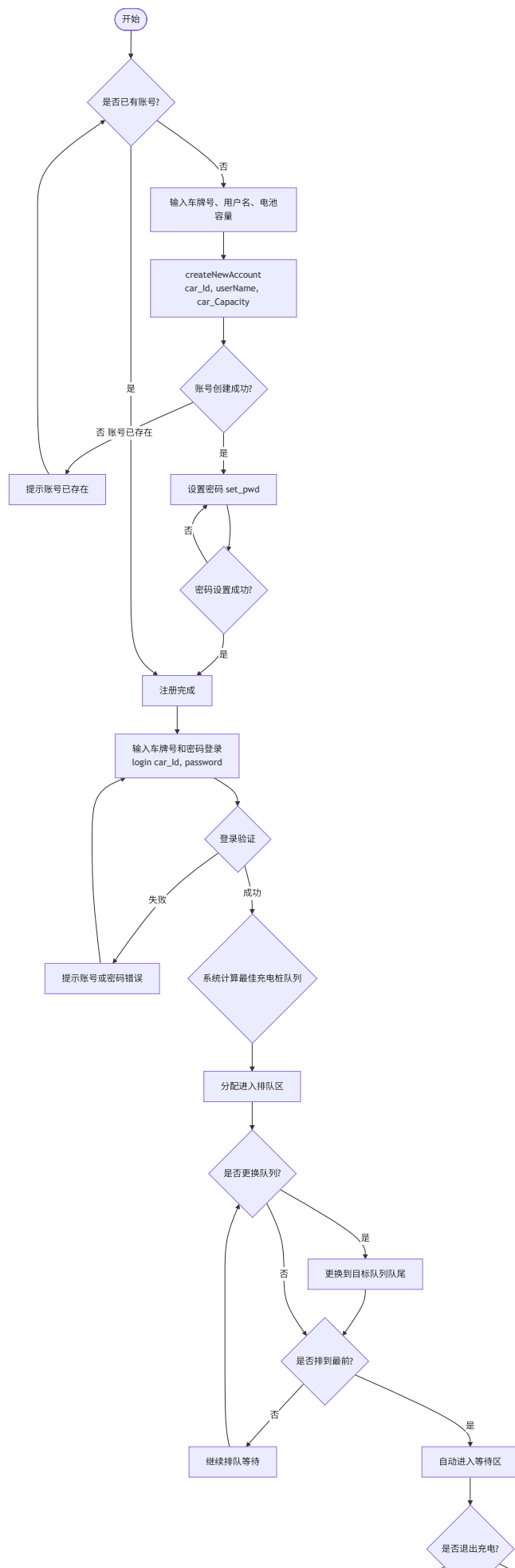
2.5.1 业务流程概述

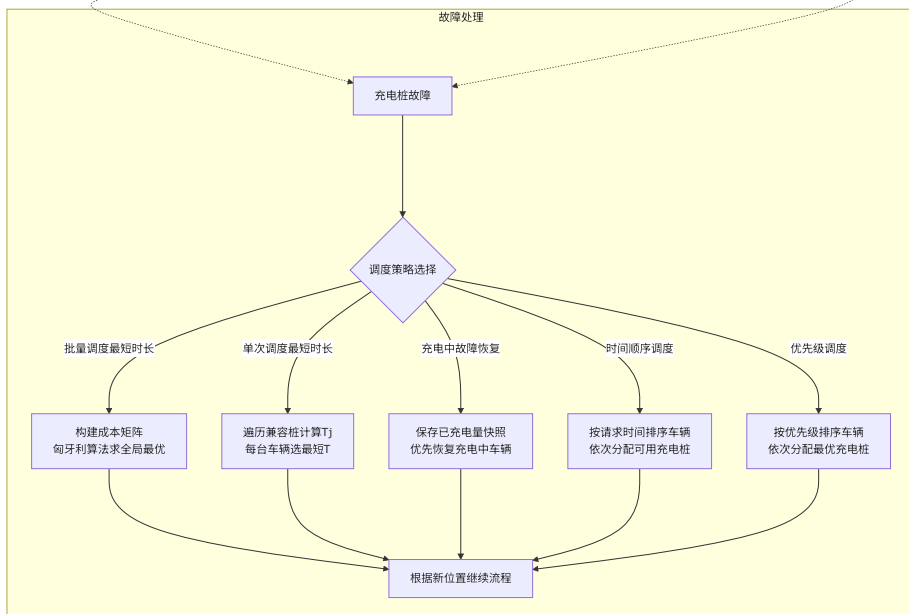
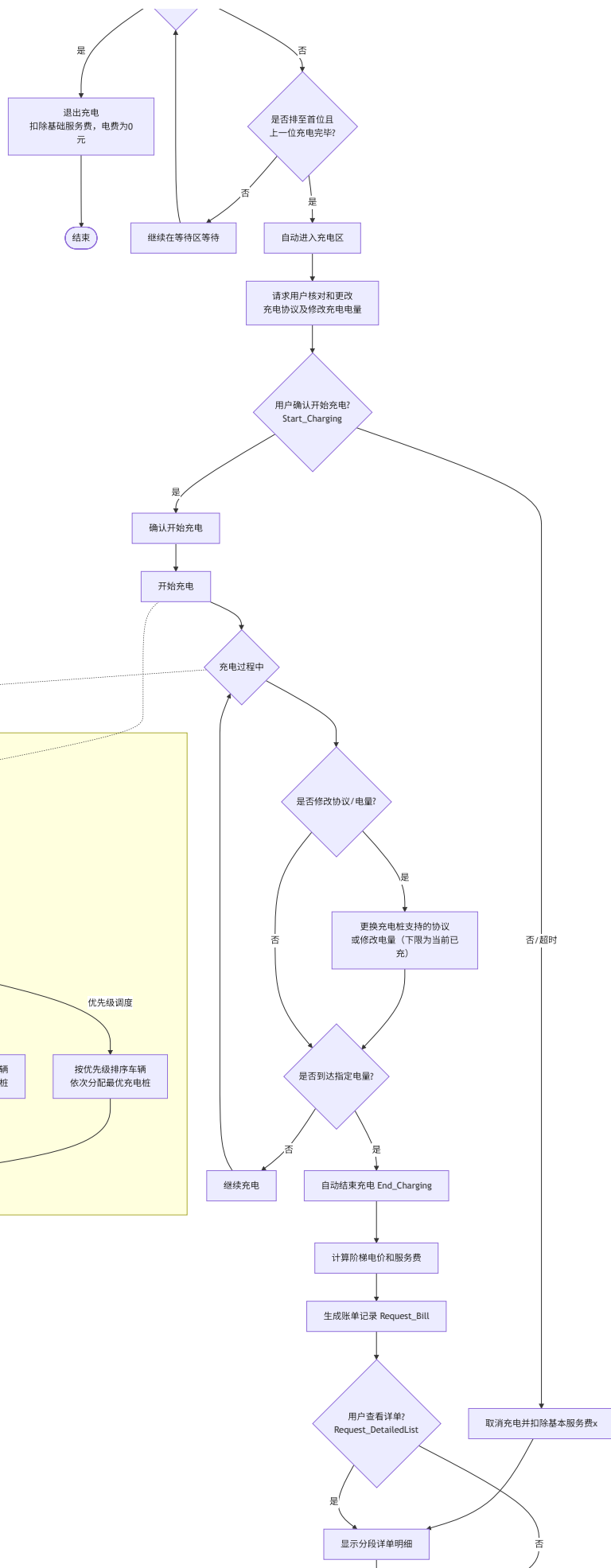
客户使用充电服务的完整业务流程从注册/登录开始，到结束一次充电服务，涵盖以下主要阶段：

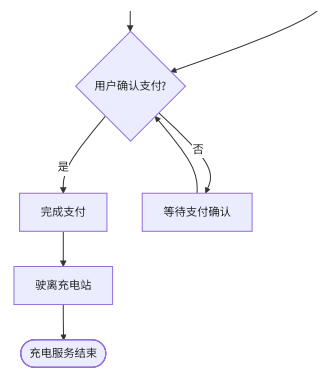
- 1. 注册/登录：新用户注册账号并设置密码，已有用户直接登录
- 2. 登录与队列分配：用户登录后，系统基于当前选定的调度策略（默认为"完成充电所需时间最短"）计算最佳充电桩队列
- 3. 排队区阶段：车辆在排队区等待，可自由更换到其他充电桩队列（退出0费用）
- 4. 等待区阶段：排队区排到最前时自动进入等待区，不可更换队列，退出需扣除基础服务费
- 5. 充电区阶段：进入充电区后请求用户核对充电协议和电量并确认开始充电（超时未确认扣除基础服务费）

6. **充电中阶段**：充电过程中可修改协议和电量
7. **充电完成与计费**：到达指定电量后自动结束充电，计算阶梯电价和服务费
8. **支付结算**：用户完成支付，驶离充电站

2.5.2 UML活动图







2.5.3 活动图流程特点总结

- 1. **注册/登录分离**：新用户需完成注册流程（创建账号+设置密码），已有用户直接登录
- 2. **用户动线简化**：排队区→等待区→充电区均为自动流转，无需用户中断确认
- 3. **调度策略可配置**：支持五种故障处理策略，系统启动参数决定默认值，管理员可运行时切换
- 4. **灵活性强**：支持中途队列更换、协议修改、电量调整等灵活操作
- 5. **容错性好**：五种独立的故障处理策略，保障不同场景下的服务连续性
- 6. **退出计费阶梯化**：排队区退出0费用，等待区和充电区确认前退出扣除基础服务费
- 7. **计费透明**：账单概览+分段详单两级查询，费用明细可追溯

三、动态结构设计

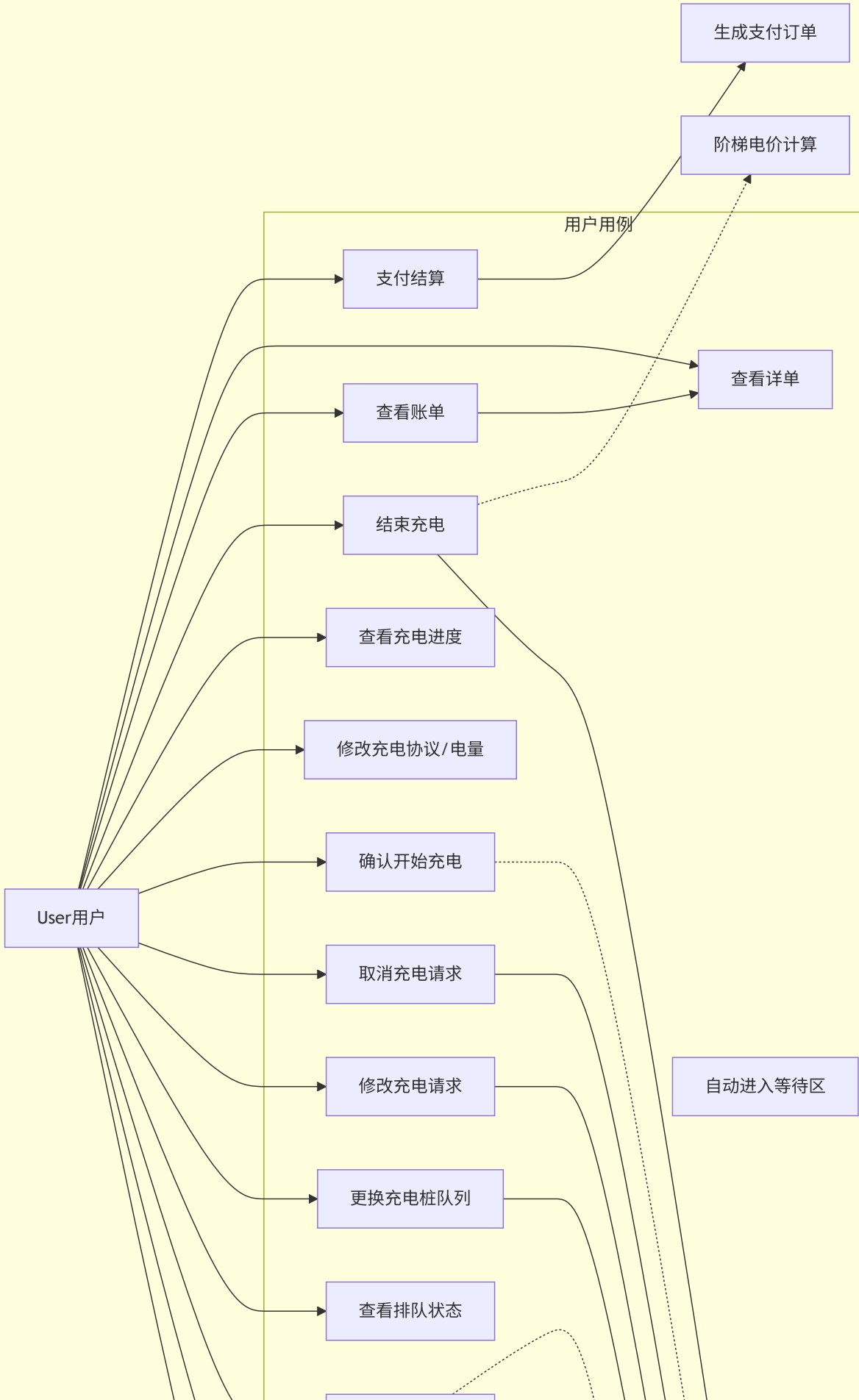
3.1 用例图

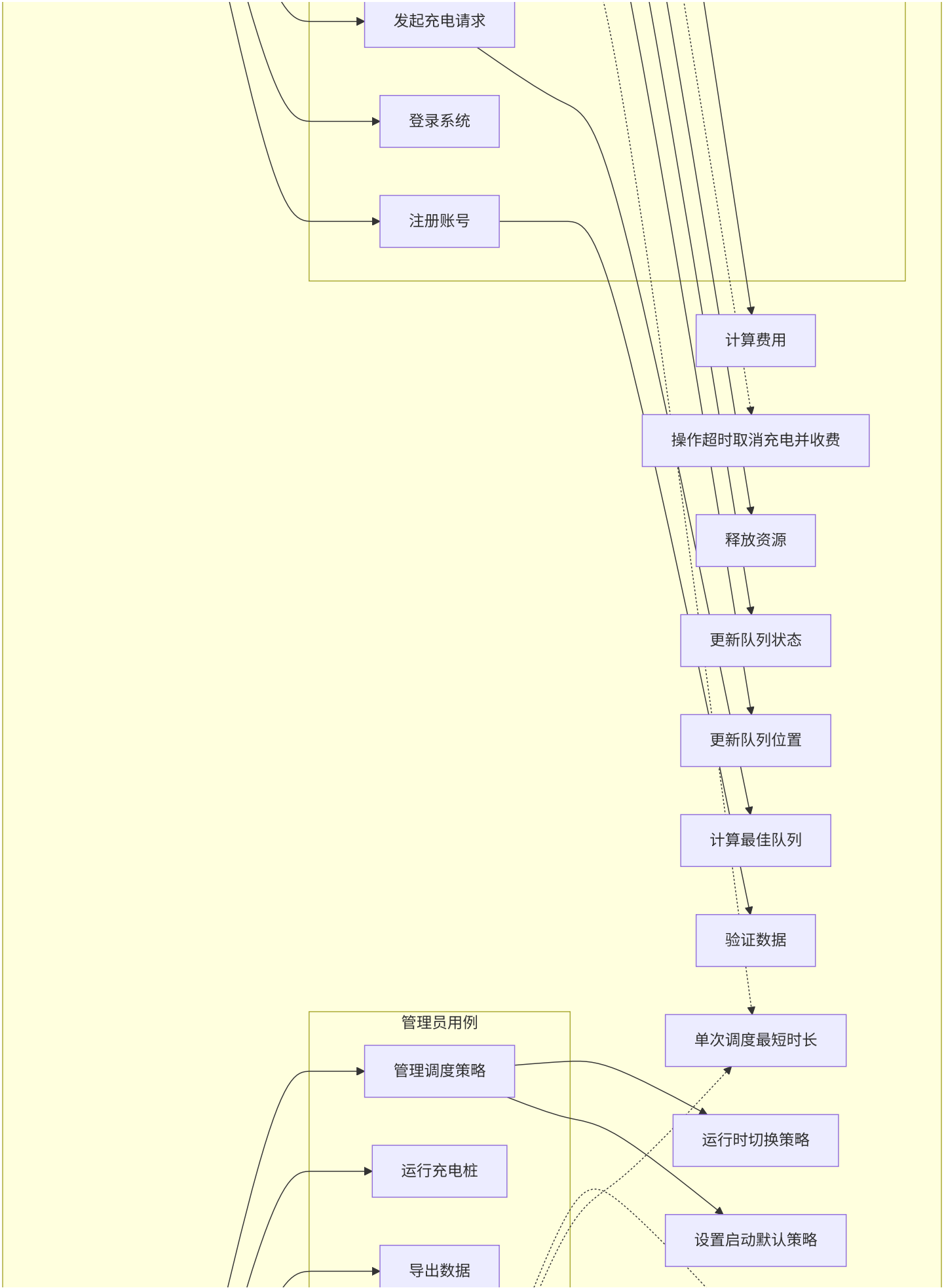
3.1.1 系统角色

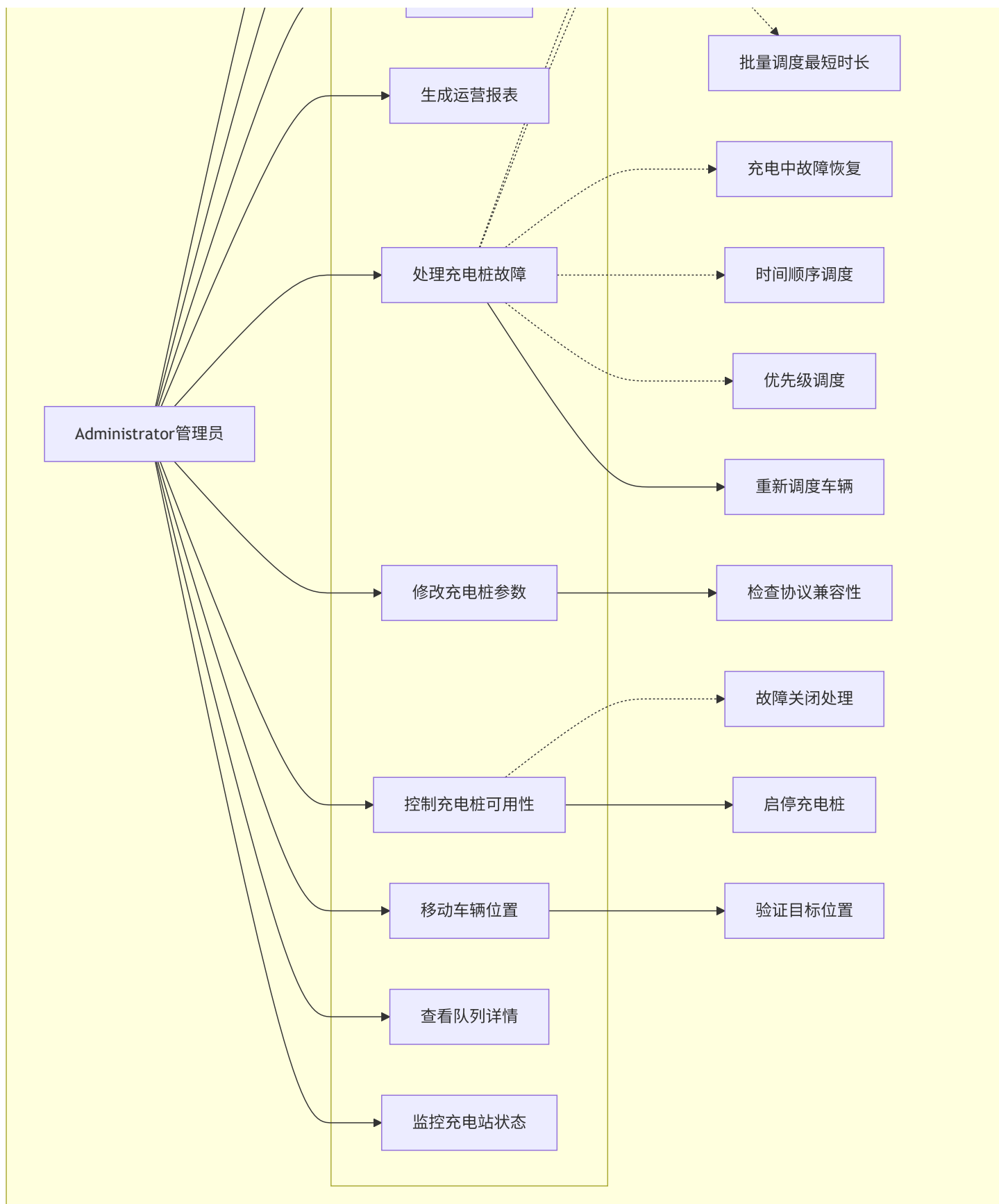
- 1. **用户 (User)**：电动汽车车主，使用充电服务
- 2. **管理员 (Administrator)**：充电站管理人员，负责系统监控与配置

3.1.2 用例图

充电站计费调度系统



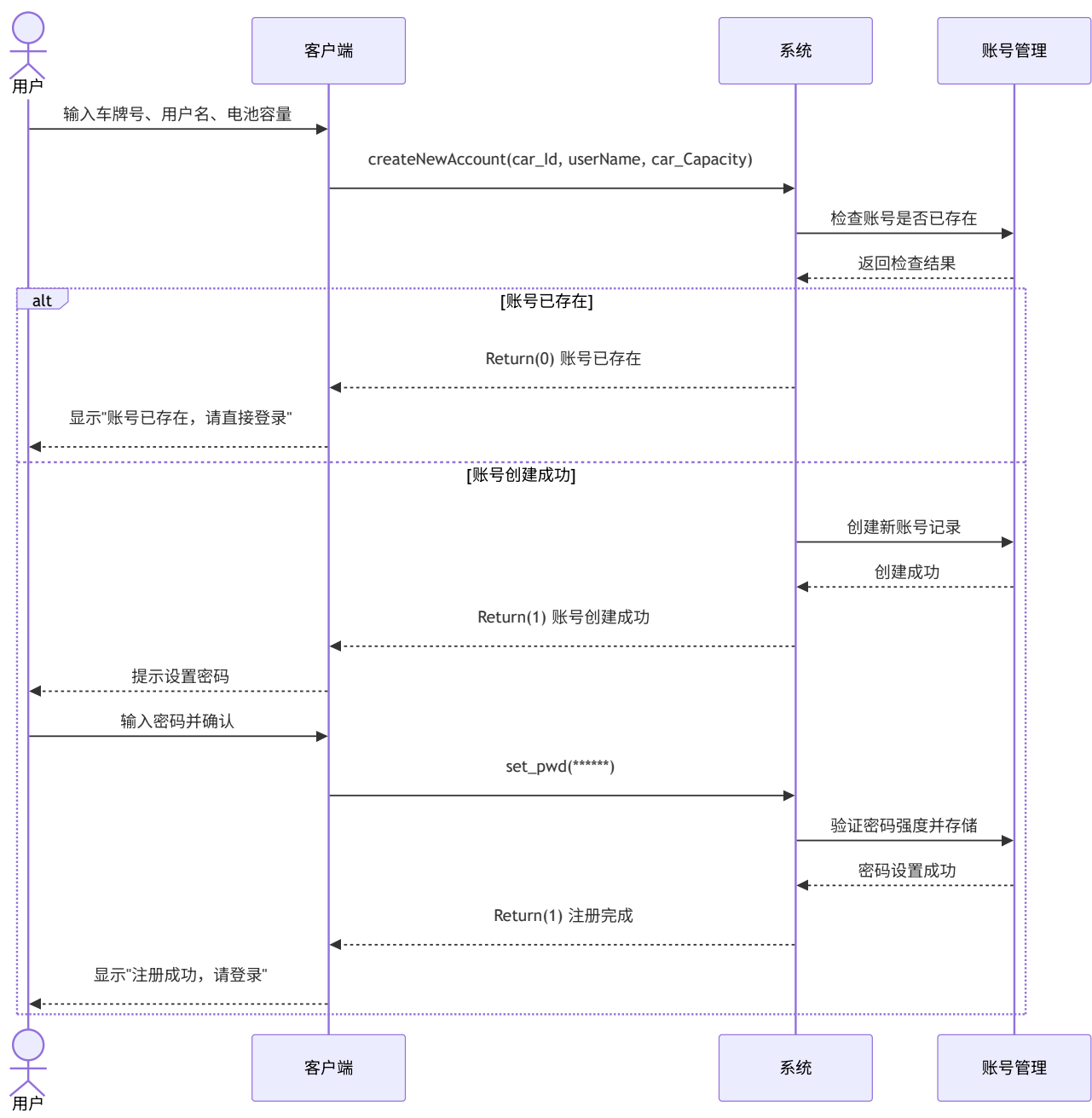




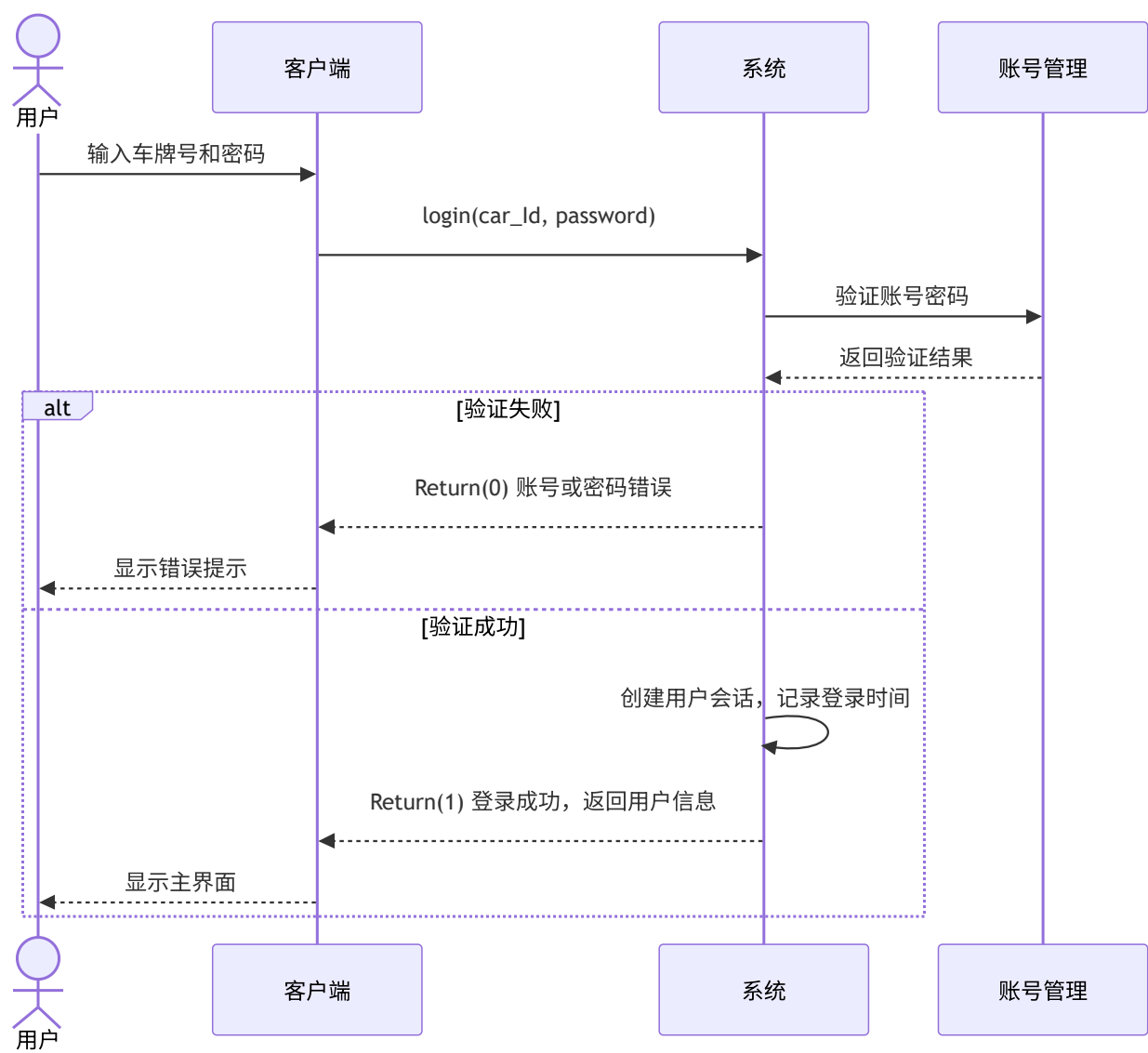
3.2 系统顺序图 (SSD)

以下为9个用例的系统顺序图。

3.2.1 注册 (UC-22)

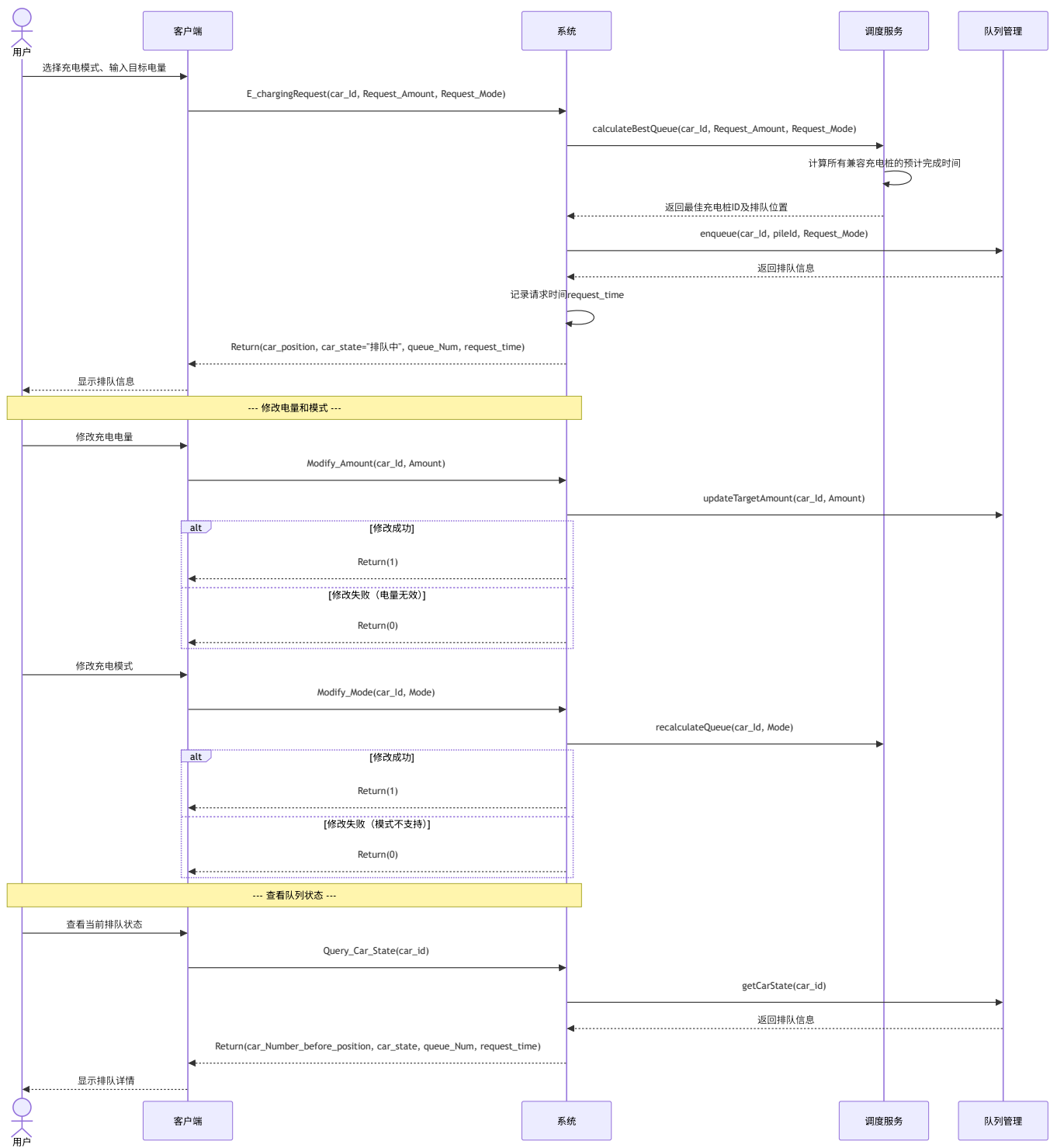


3.2.2 登录 (UC-01)

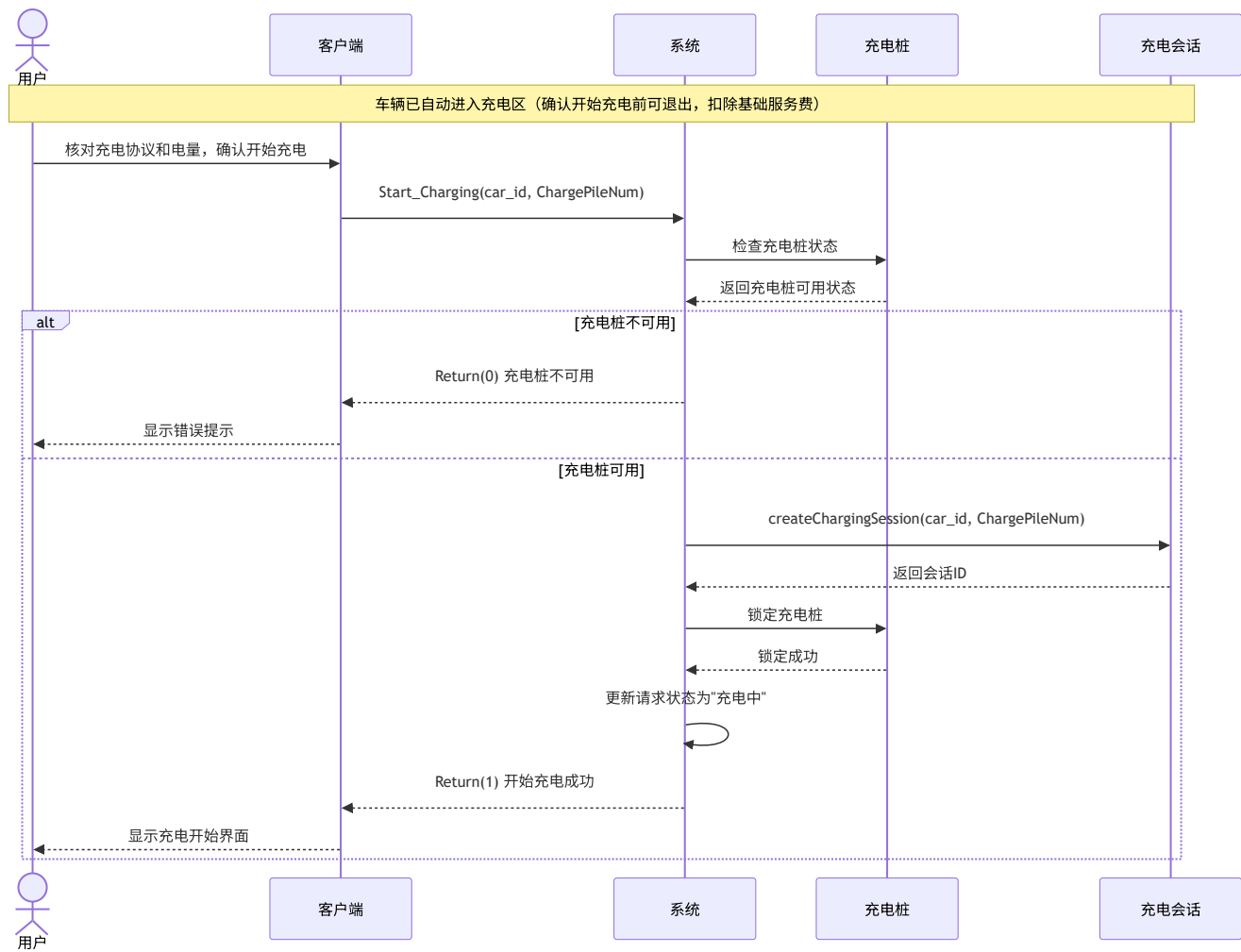


3.2.3 充电申请 (UC-02、UC-03、UC-05、UC-08、UC-10、UC-11)

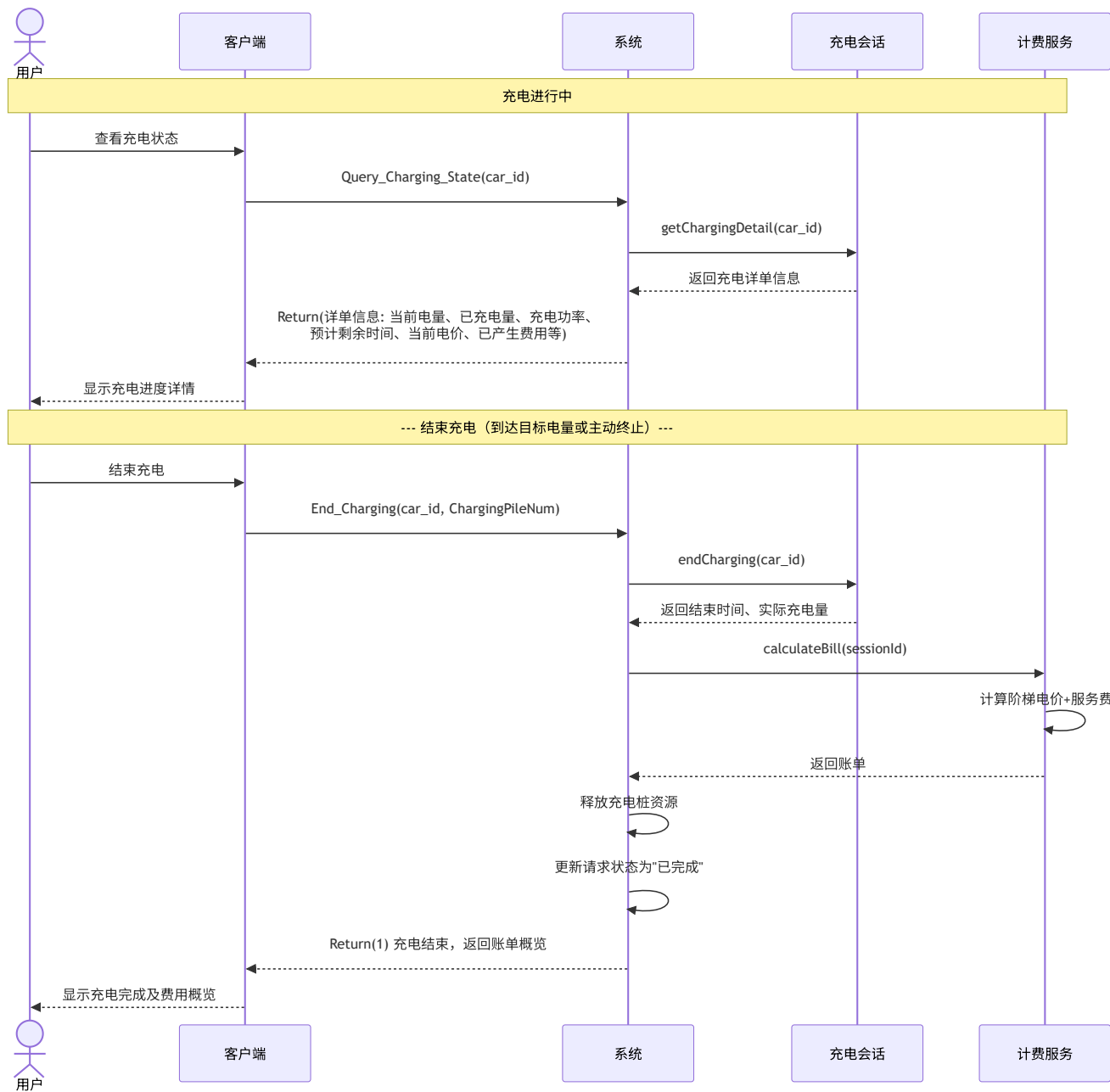
阶段一：提交充电申请与查看队列状态



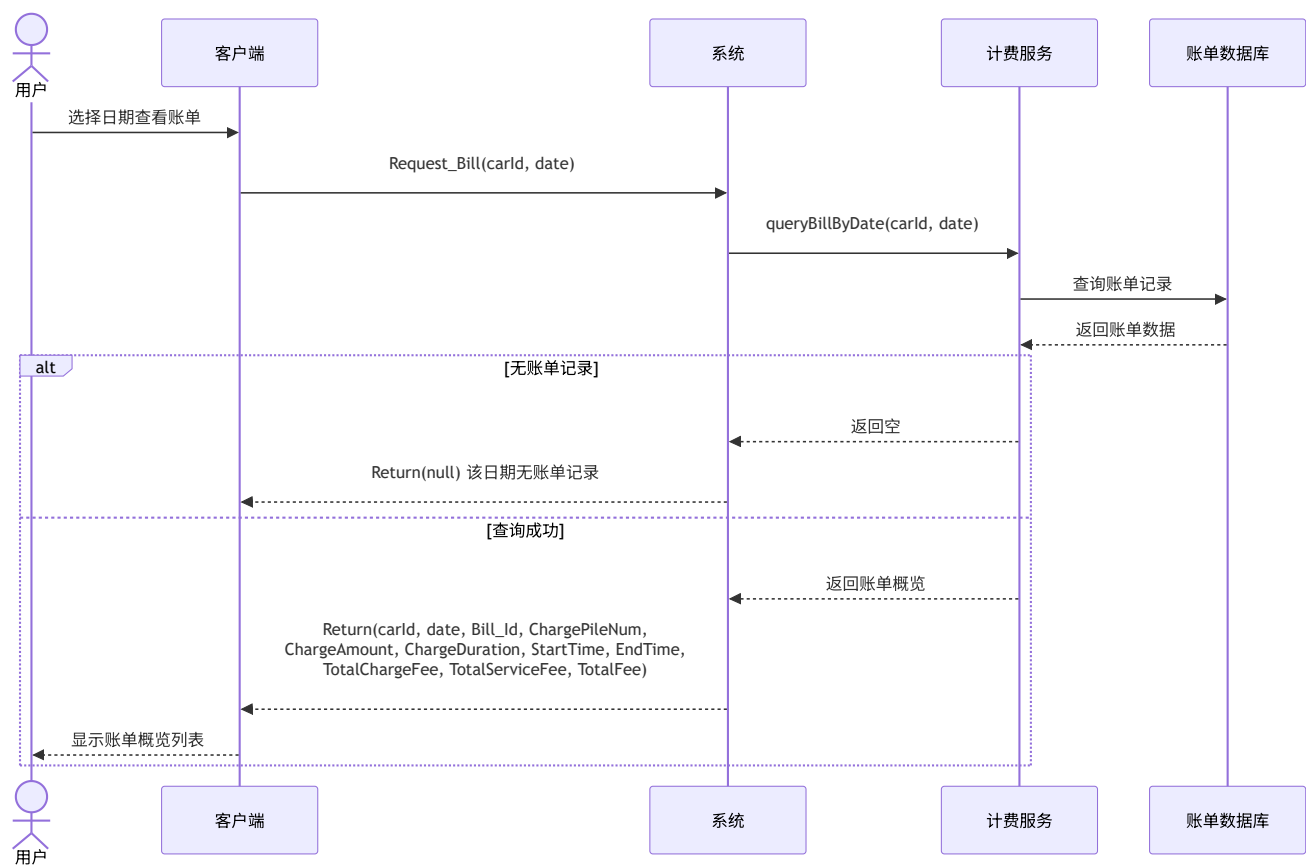
阶段二：开始充电



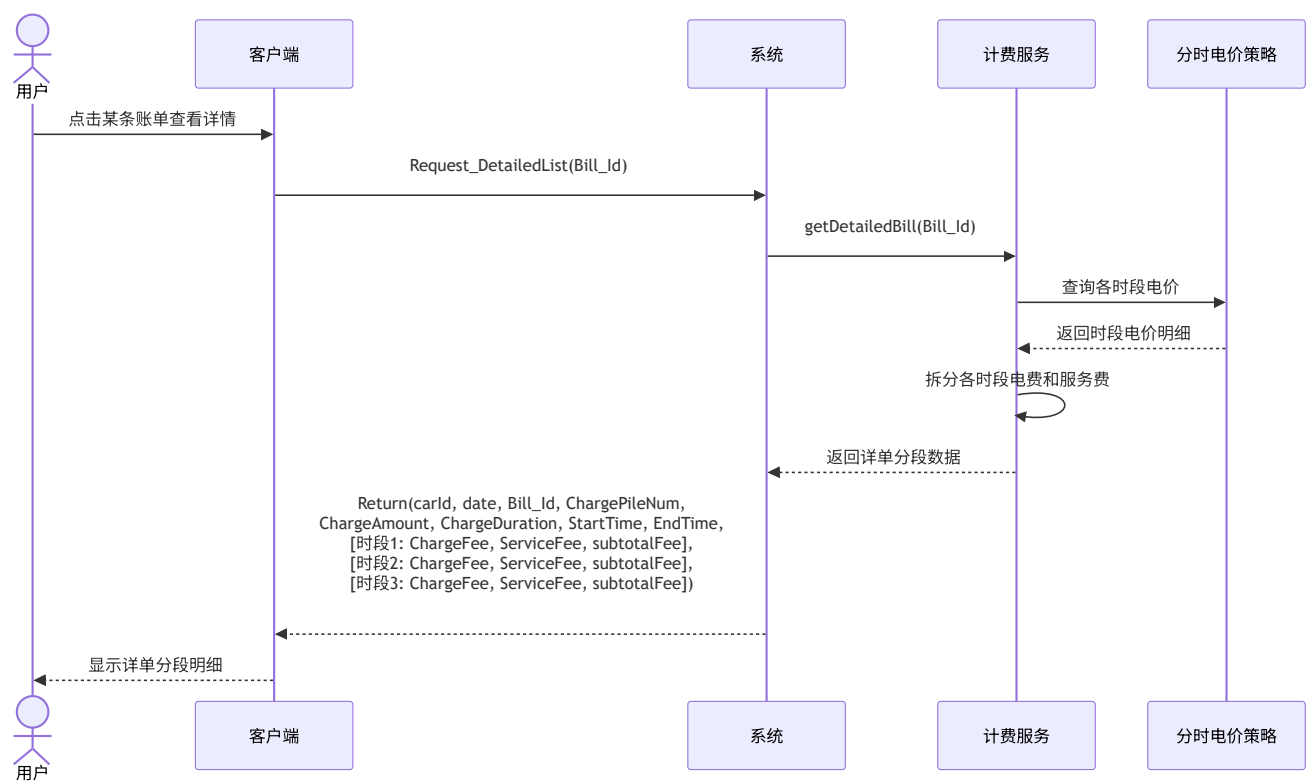
阶段三：查看充电状态与结束充电



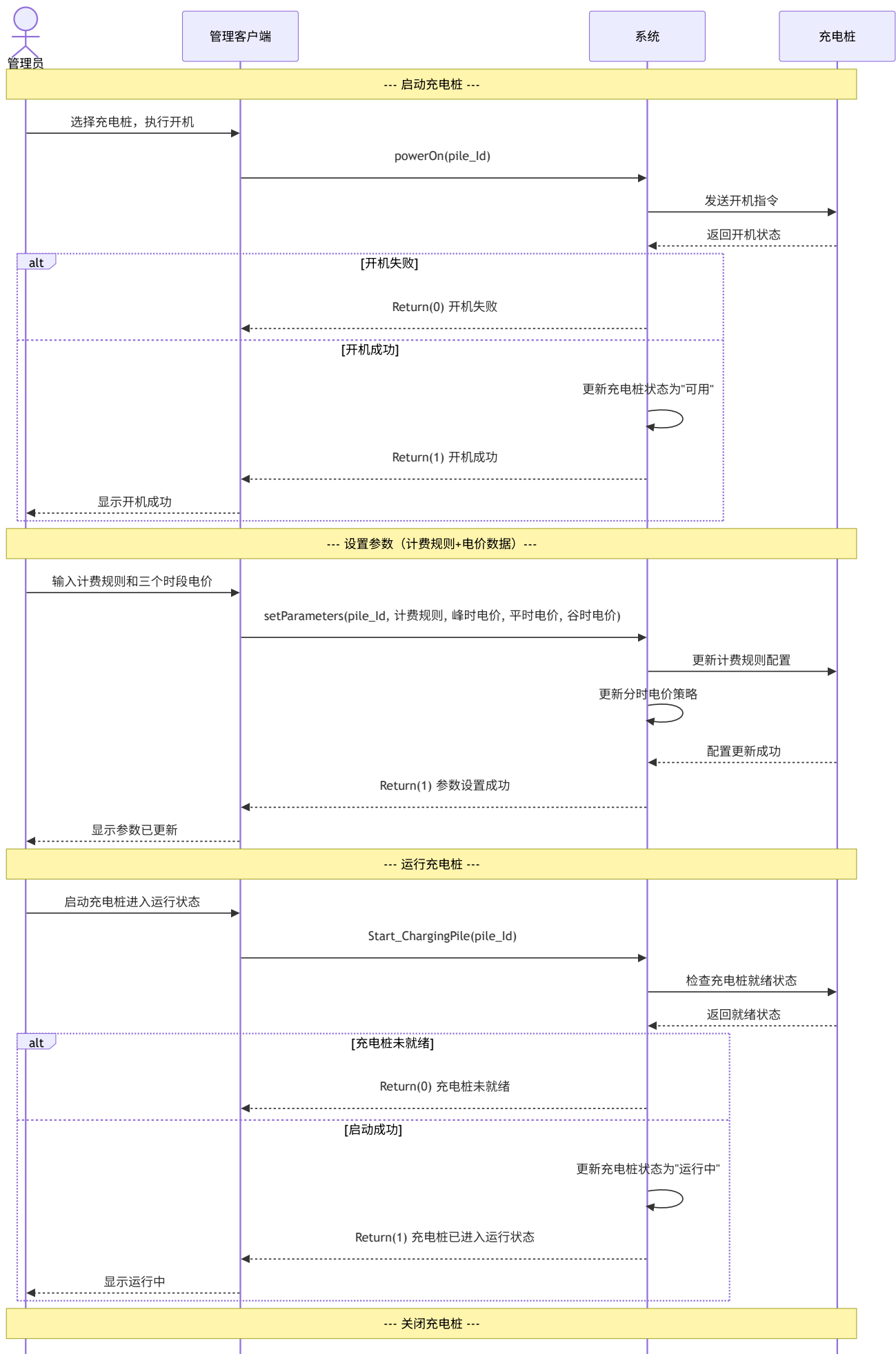
3.2.4 查看账单（UC-12）

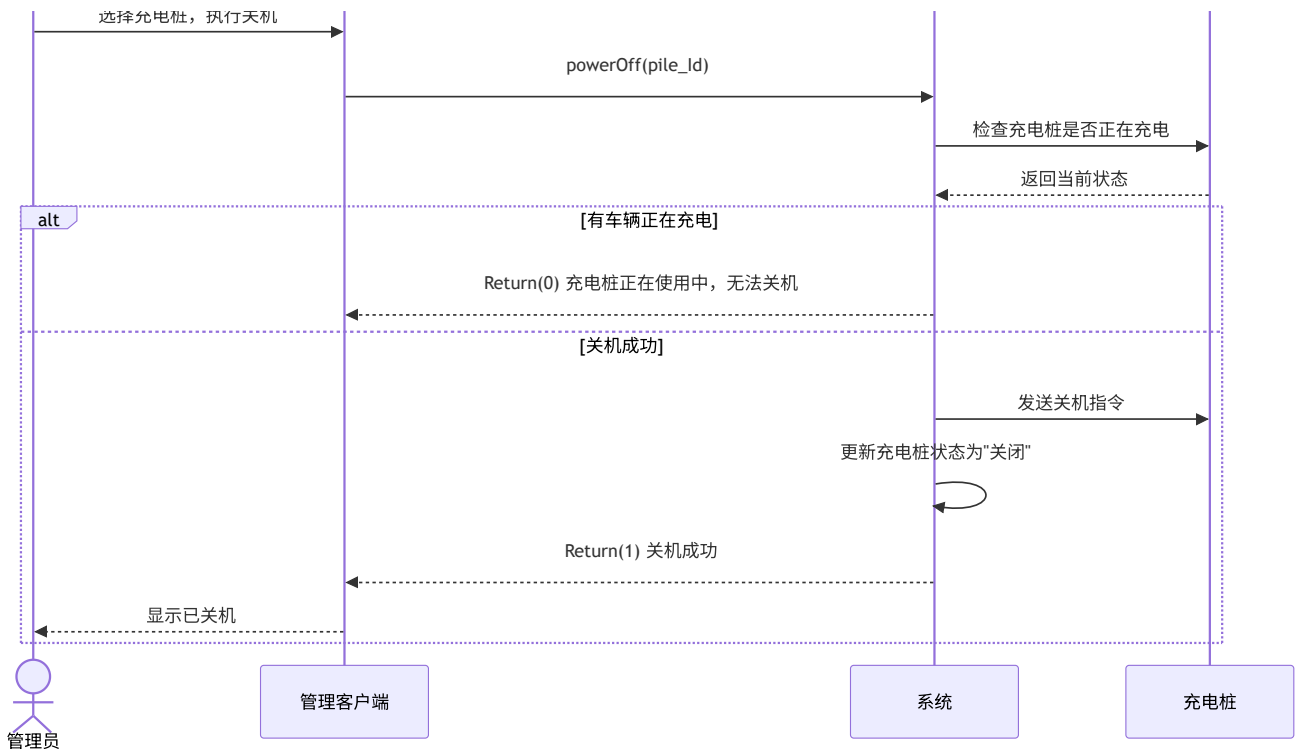


3.2.5 查看详细单（UC-12a）

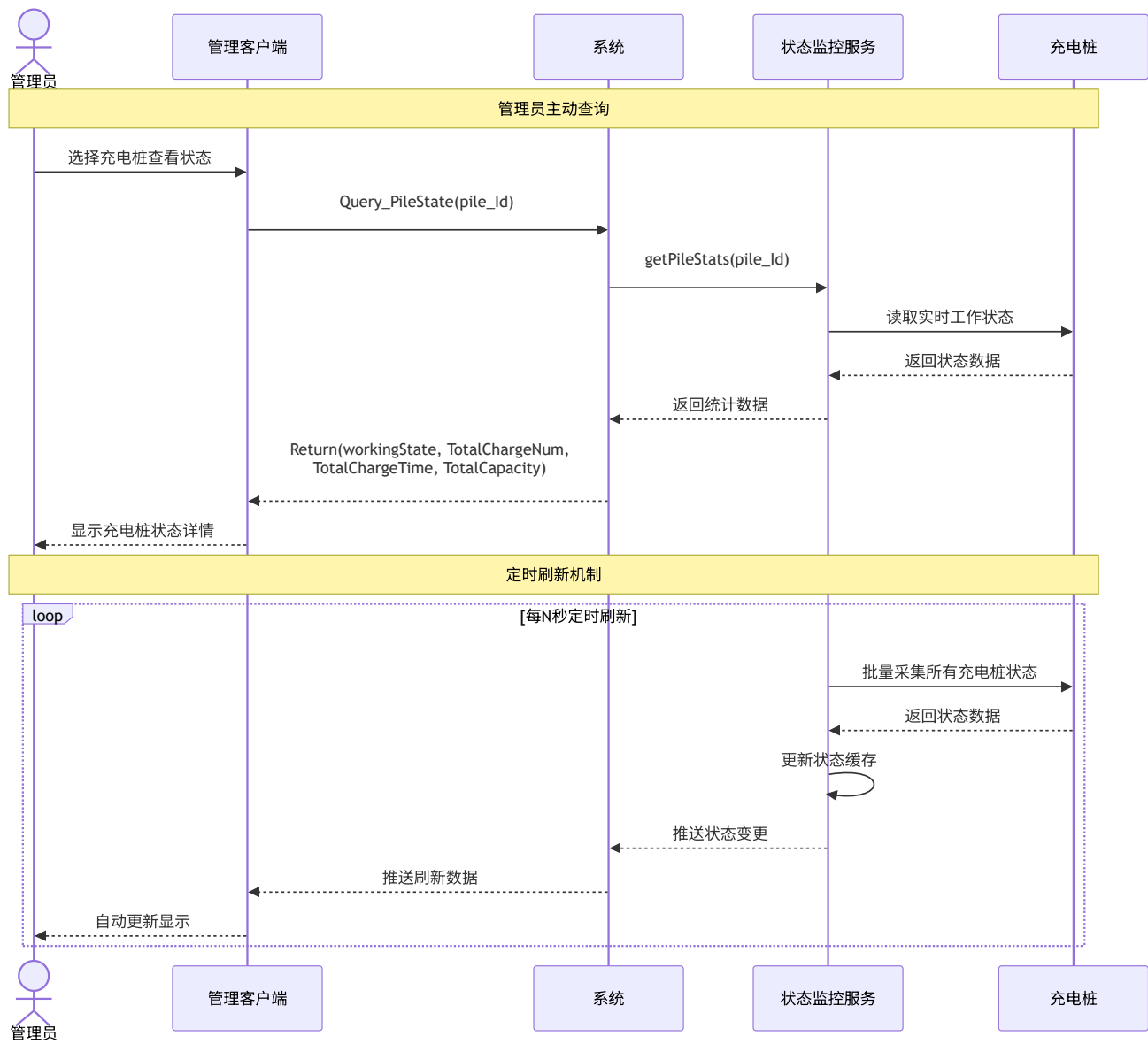


3.2.6 运行充电桩（UC-23、UC-17、UC-18）

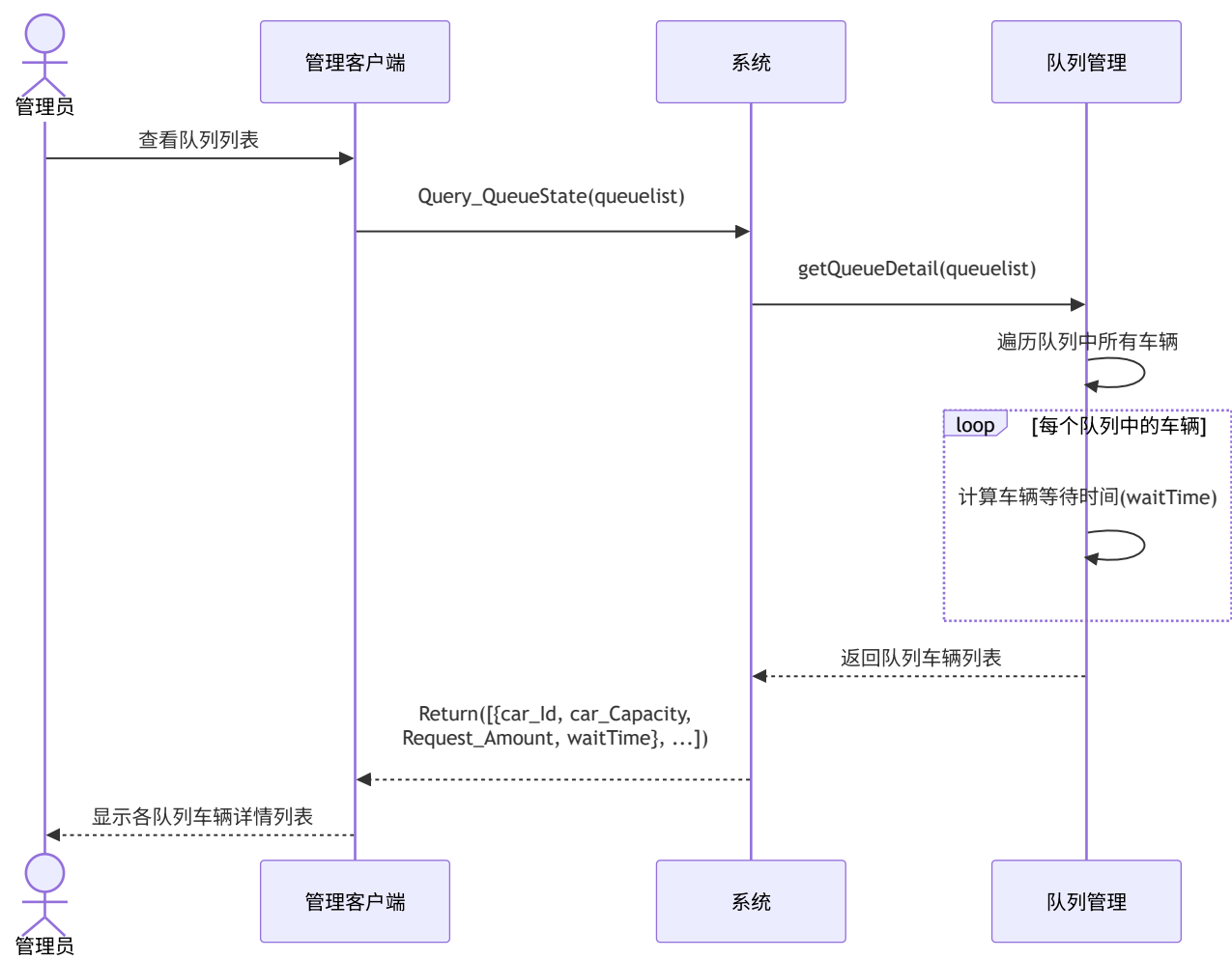




3.2.7 查看充电桩状态（UC-14）

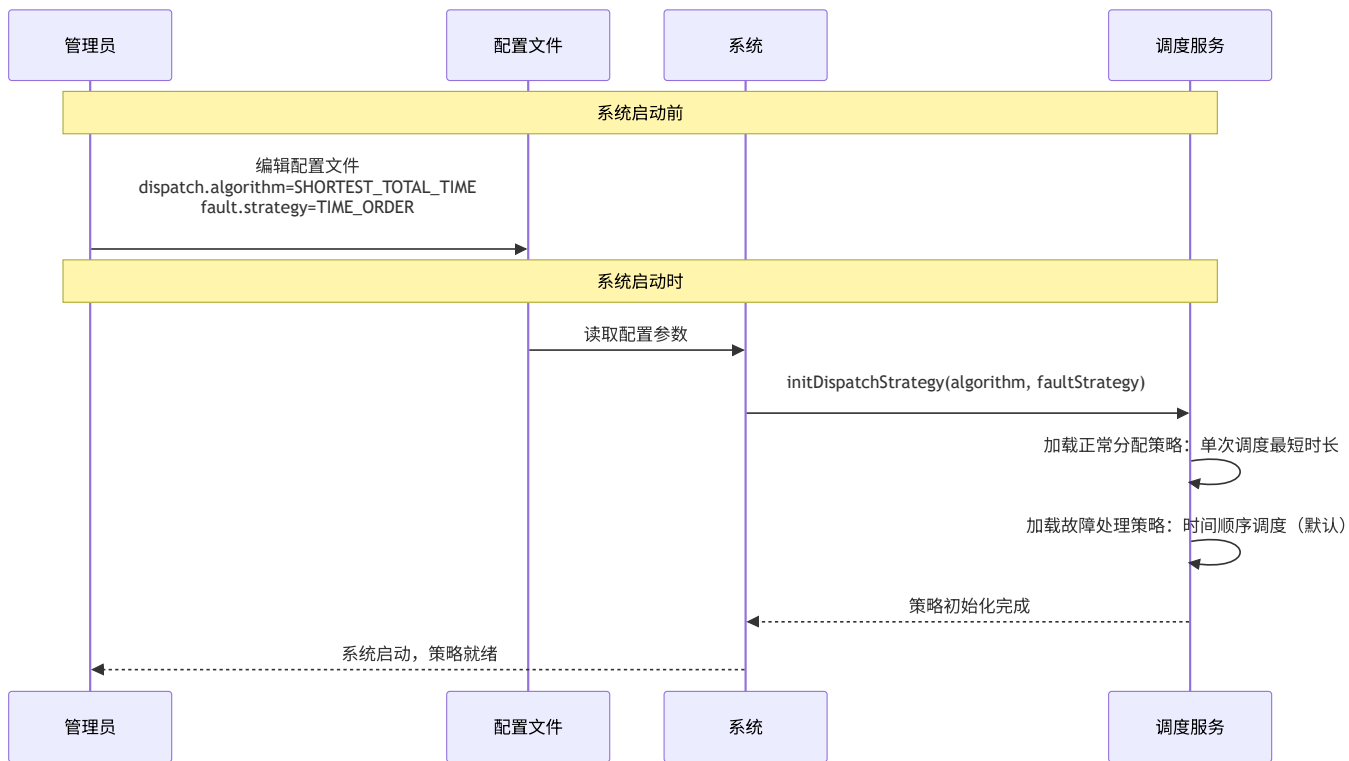


3.2.8 查看队列状态 (UC-15)

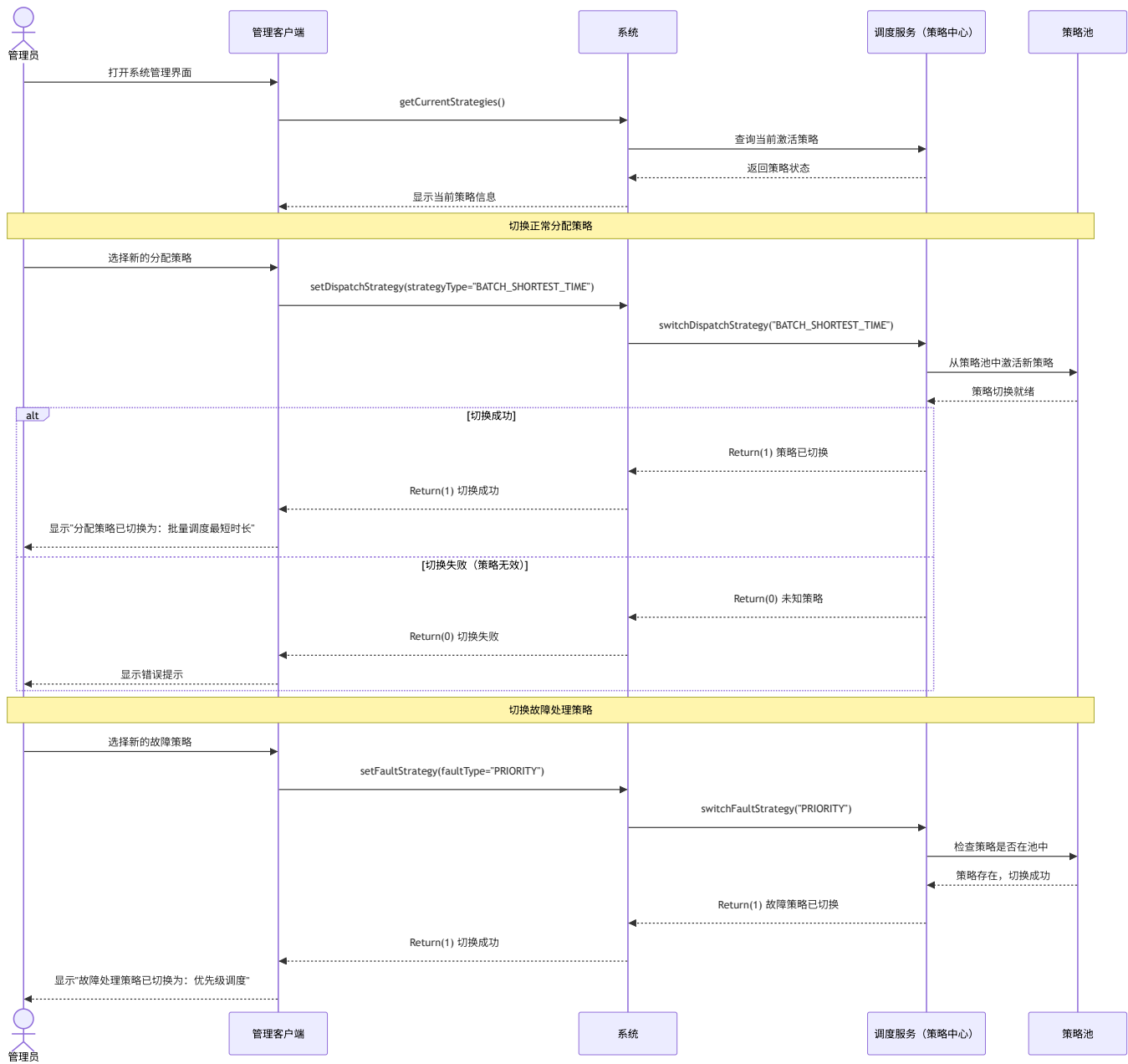


3.2.9 管理调度策略 (UC-41、UC-42、UC-43)

系统启动时设置默认调度策略



管理员运行时切换策略



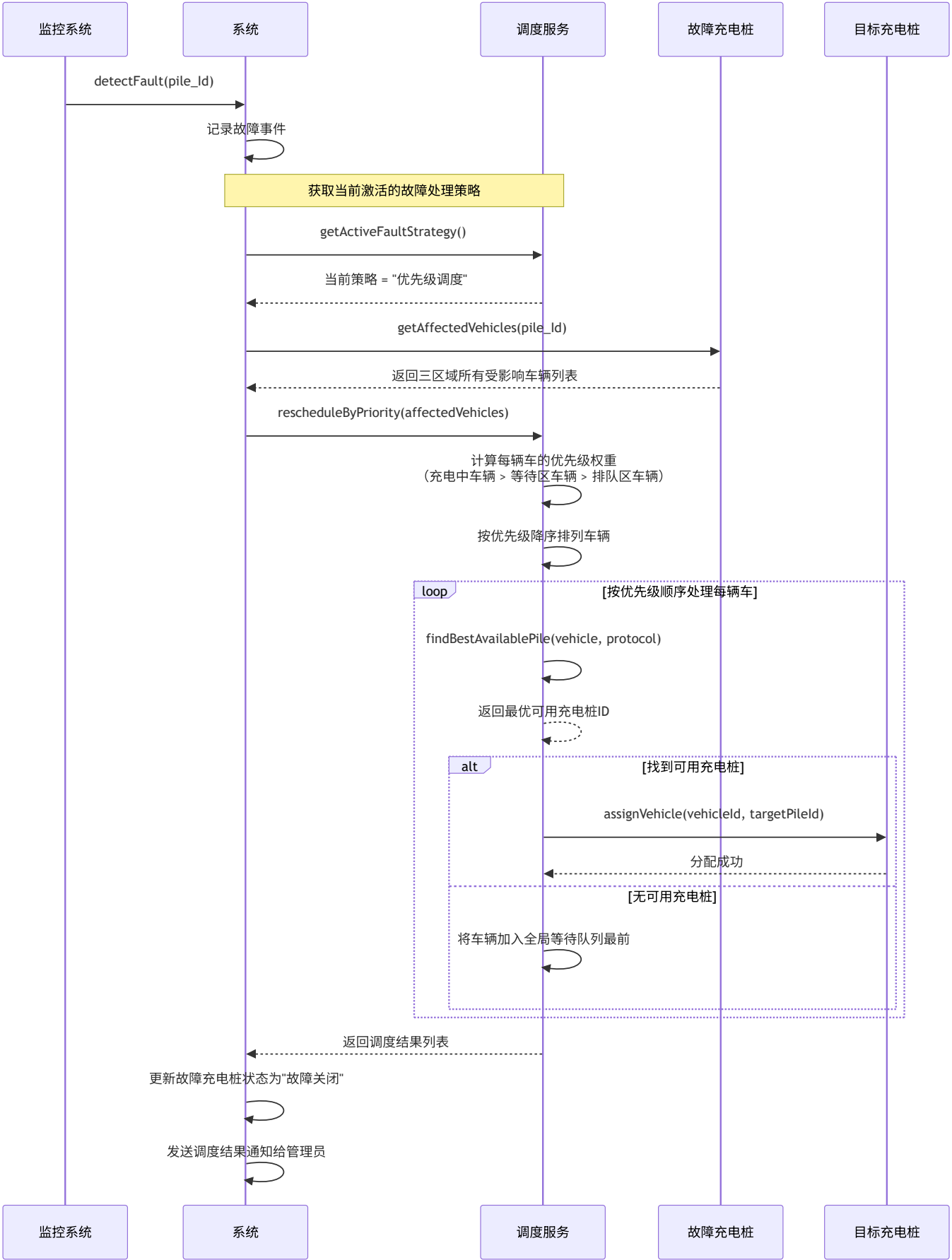
可用策略一览

策略类型	策略名称	标识符	适用场景
正常分配	单次调度最短时长	SHORTEST_TOTAL_TIME	默认。单台车辆提交申请时，遍历所有兼容桩选总完成时间最短
正常分配	批量调度最短时长	BATCH_SHORTEST_TIME	多台车辆同时分配时，匈牙利算法求全局最优
故障处理	优先级调度	PRIORITY	故障时按区域+会员+等待时间排序后分配
故障处理	时间顺序调度	TIME_ORDER	故障时按请求时间先来后到排序
故障处理	充电中故障恢复	FAULT_RECOVERY	充电中故障时保存快照，优先恢复充电中车辆
故障处理	单次调度最短时长	SHORTEST_TOTAL_TIME	故障时用贪心算法为每台车选最优桩
故障处理	批量调度最短时长	BATCH_SHORTEST_TIME	故障时用匈牙利算法批量求全局最优分配

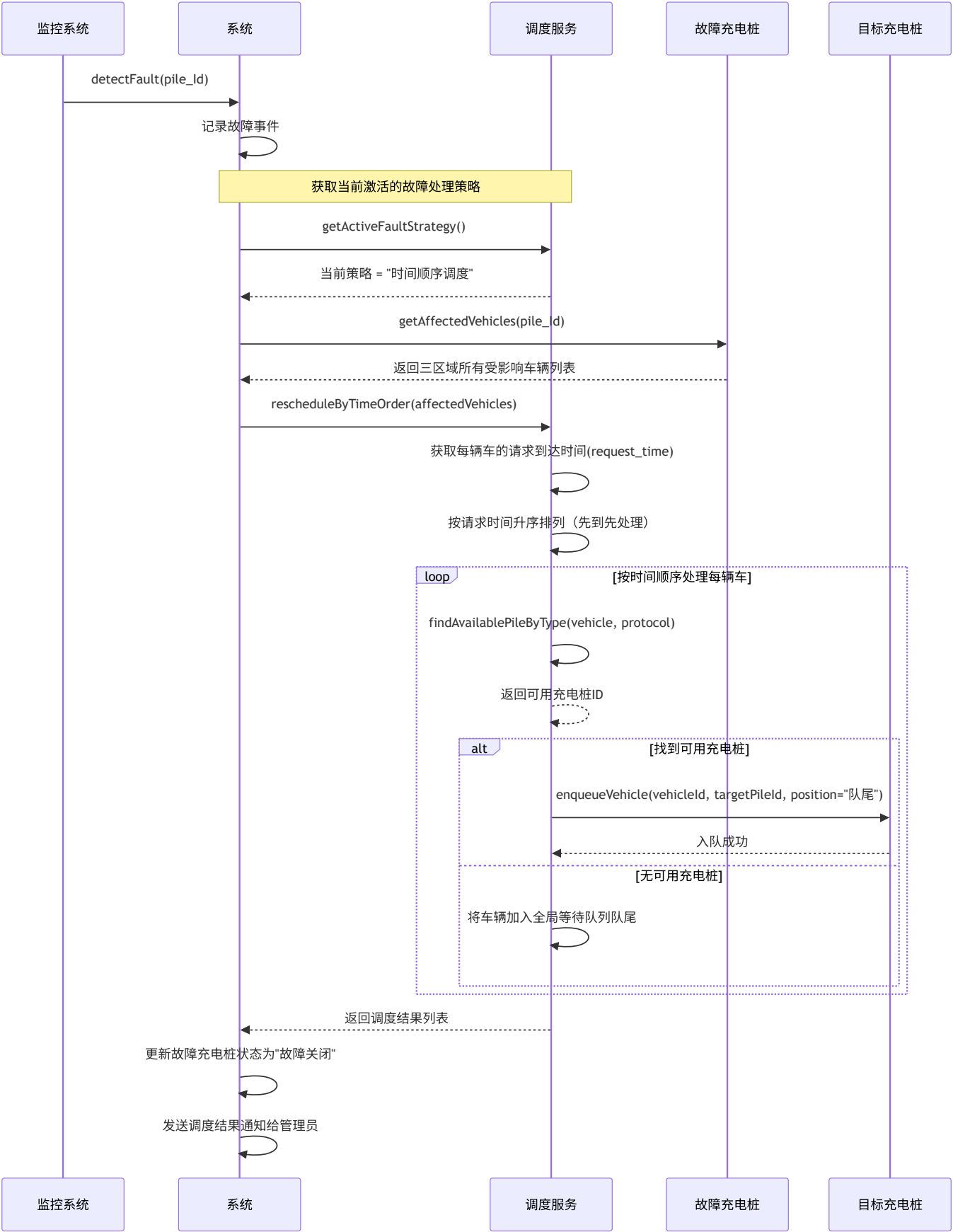
3.3 故障调度交互图

以下为五种故障处理交互过程。策略由系统启动参数决定默认值，管理员运行期间可动态切换。

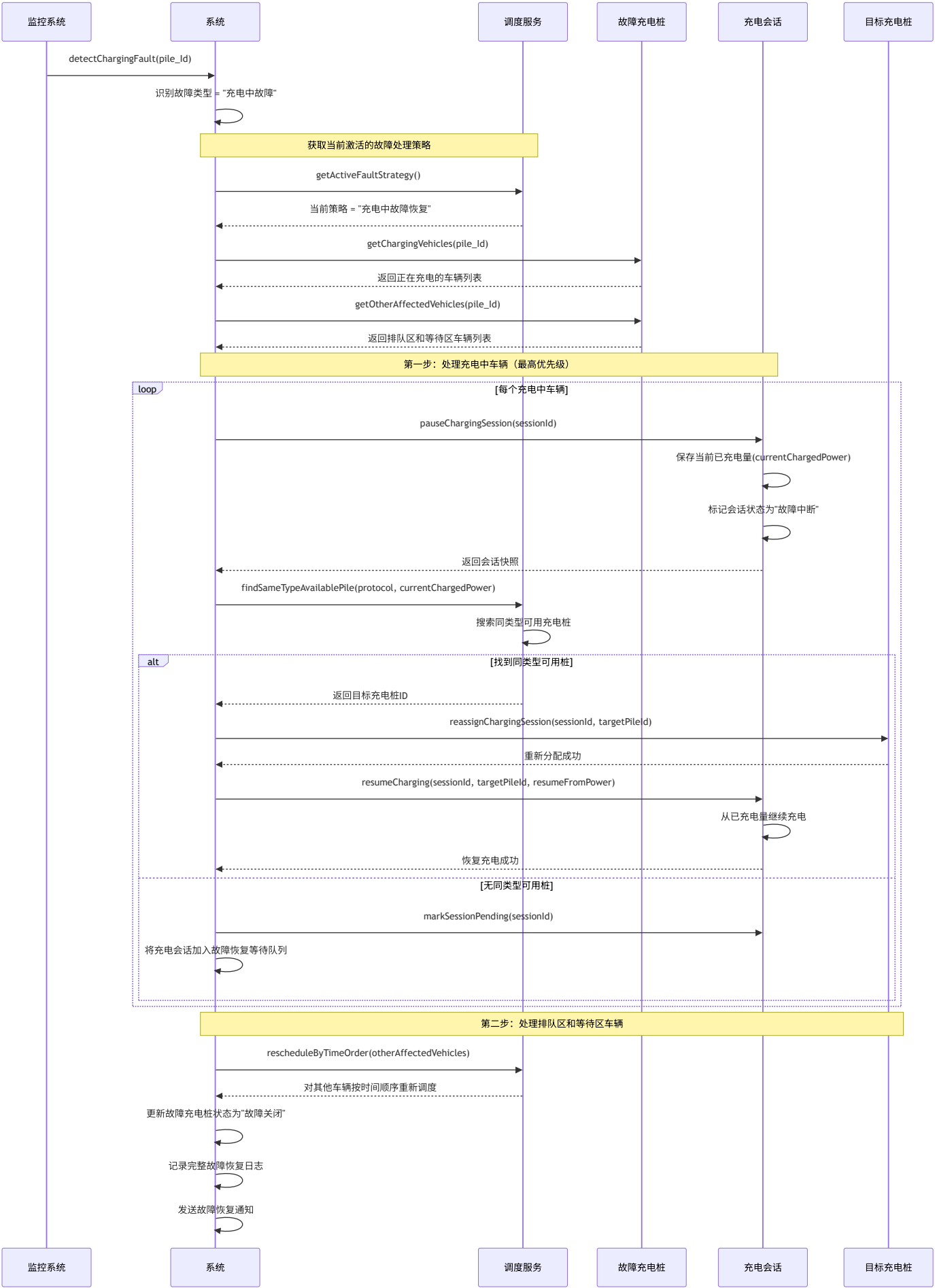
3.3.1 优先级调度（SSD-F1）



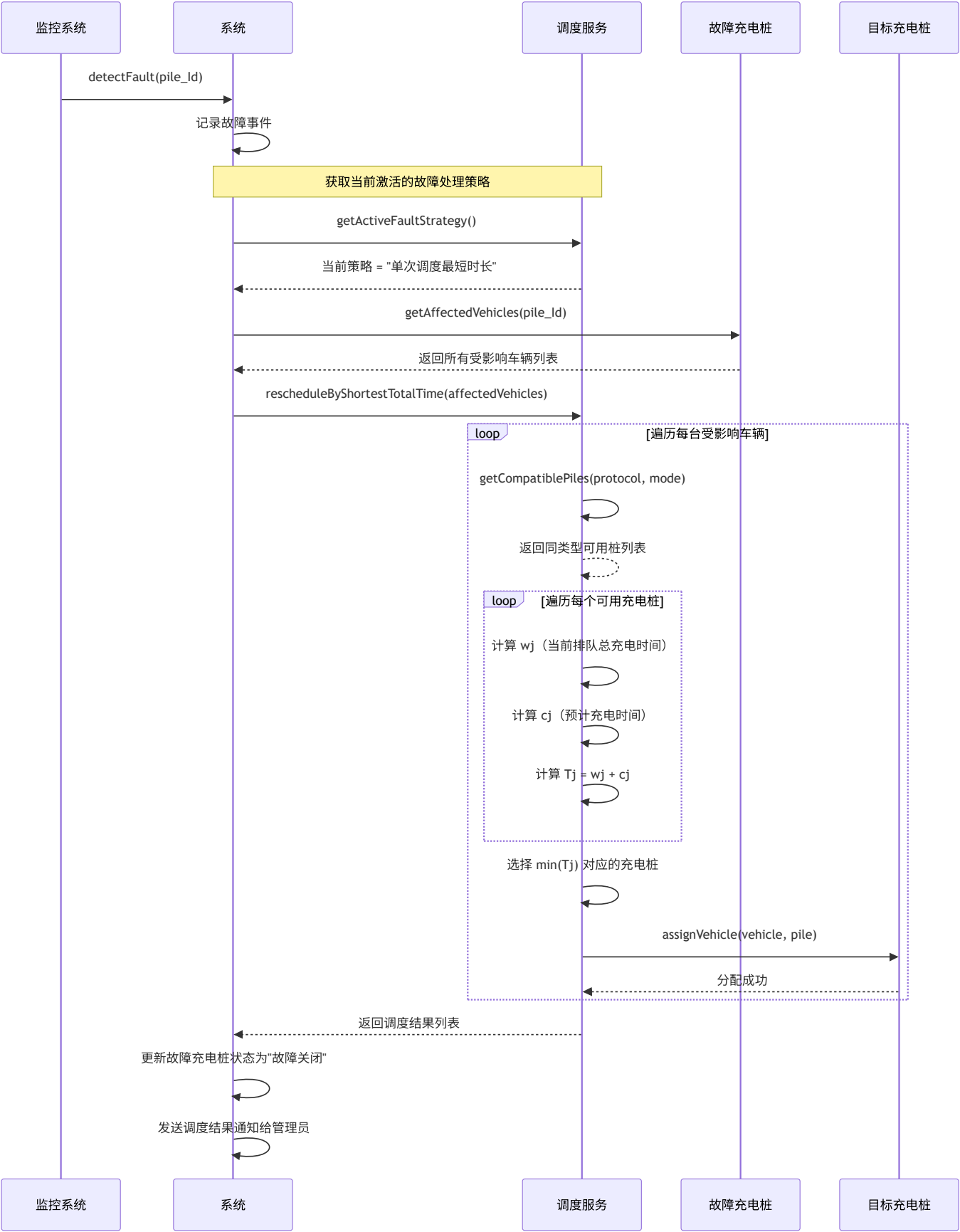
3.3.2 时间顺序调度（SSD-F2）



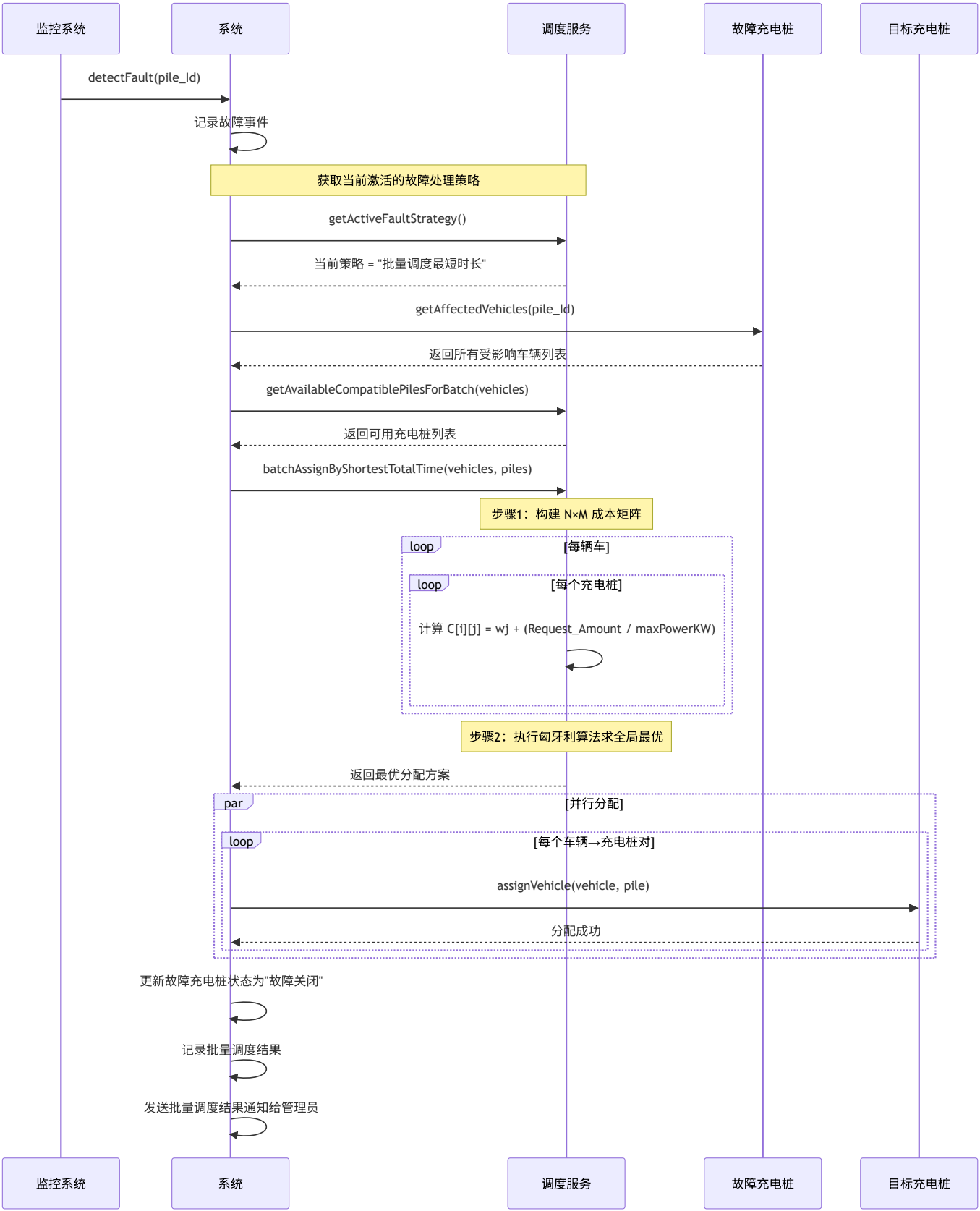
3.3.3 充电中故障恢复 (SSD-F3)



3.3.4 单次调度最短时长 — 故障场景（SSD-F4）



3.3.5 批量调度最短时长 — 故障场景（SSD-F5）



3.4 操作契约

3.4.1 用户相关操作契约

契约1: createNewAccount(car_Id, userName, car_Capacity)

- **操作:** createNewAccount(car_Id: String, userName: String, car_Capacity: Double)
- **交叉引用:** UC-22 注册账号
- **前置条件:**
 - i. 用户已打开客户端应用
 - ii. car_Id 格式正确 (车牌号)
 - iii. car_Capacity > 0
- **后置条件:**
 - i. 创建新的 User 对象 (userName, car_Id, car_Capacity)
 - ii. 创建关联的 Vehicle 对象 (car_Id, car_Capacity)
 - iii. 用户状态初始化为"未激活" (待设置密码)
- **异常处理:**
 - car_Id 已存在: Return(0), 提示"账号已存在, 请直接登录"
 - car_Capacity ≤ 0: Return(0), 提示"电池容量无效"

契约2: set_pwd(password)

- **操作:** set_pwd(password: String)
- **交叉引用:** UC-22 注册账号 (子步骤)
- **前置条件:**
 - i. createNewAccount 已成功执行
 - ii. 用户对象状态为"未激活"
 - iii. 密码符合强度要求
- **后置条件:**
 - i. User.password 被设置 (加密存储)
 - ii. 用户状态更新为"已激活"
- **异常处理:**
 - 密码强度不足: Return(0), 提示密码要求
 - 用户不存在: Return(0)

契约3: login(car_Id, password)

- **操作:** login(car_Id: String, password: String)
- **交叉引用:** UC-01 登录系统
- **前置条件:**
 - i. 用户已注册且账号状态为"已激活"
 - ii. 用户未登录
- **后置条件:**
 - i. 验证账号密码匹配
 - ii. 创建用户会话
 - iii. 记录用户登录时间
- **异常处理:**
 - 账号不存在或密码错误: Return(0)

- 账号已被锁定：Return(0)，提示联系管理员

契约4：E_chargingRequest(car_Id, Request_Amount, Request_Mode)

- **操作：** E_chargingRequest(car_Id: String, Request_Amount: Double, Request_Mode: ChargeMode)
- **交叉引用：** UC-02 发起充电请求
- **前置条件：**
 - i. 用户已登录
 - ii. Request_Amount > 当前电量
 - iii. Request_Mode 有效（快充/慢充）
- **后置条件：**
 - i. 创建 ChargingRequest 对象
 - ii. 系统基于当前激活的分配策略计算最佳充电桩
 - iii. 车辆进入对应充电桩的排队区
 - iv. 记录 request_time
 - v. 返回 car_position, car_state="排队中", queue_Num, request_time
- **异常处理：**
 - Request_Amount 无效：Return 错误信息
 - 无兼容充电桩：Return(0) 无可用资源

契约4a：assignByShortestTotalTime(car_Id, Request_Amount, Request_Mode)（正常分配）

- **操作：** assignByShortestTotalTime(car_Id: String, Request_Amount: Double, Request_Mode: ChargeMode)
- **交叉引用：** UC-02 发起充电请求（单次调度最短时长策略）
- **前置条件：**
 - i. 车辆已登录且有效
 - ii. Request_Amount > 0
 - iii. Request_Mode 有效
 - iv. 系统当前分配策略为 SHORTEST_TOTAL_TIME（默认）或 BATCH_SHORTEST_TIME
- **后置条件：**
 - i. 获取所有兼容 Request_Mode 的充电桩列表
 - ii. 对每个充电桩计算 w_i （当前排队等待时间）和 c_i （预计充电时间）
 - iii. 计算 $T_i = w_i + c_i$
 - iv. 选择 $\min(T_i)$ 对应的充电桩作为目标桩
 - v. 车辆进入目标桩排队区
 - vi. 返回分配结果和预计完成时间
- **异常处理：**
 - 无兼容充电桩：返回无可用资源提示
 - 所有充电桩故障（全部 $T_i = \infty$ ）：返回系统繁忙提示

契约5：Modify_Amount(car_Id, Amount)

- **操作：** Modify_Amount(car_Id: String, Amount: Double)
- **交叉引用：** UC-05 修改充电请求
- **前置条件：**
 - i. 充电请求存在且状态为"排队中"或"等待中"
 - ii. Amount > 当前电量
- **后置条件：**

- i. 更新 ChargingRequest.targetPowerKWh
- ii. 重新计算预计充电时长
- **异常处理:**
 - 请求不存在: Return(0)
 - Amount 无效: Return(0)

契约6: Modify_Mode(car_Id, Mode)

- **操作:** Modify_Mode(car_Id: String, Mode: ChargeMode)
- **交叉引用:** UC-05 修改充电请求
- **前置条件:**
 - i. 充电请求存在且状态为"排队中"
 - ii. Mode 有效
- **后置条件:**
 - i. 更新 ChargingRequest.chargingMode
 - ii. 从原队列移除, 加入新模式对应队列队尾
 - iii. 重新计算排队位置
- **异常处理:**
 - 请求不存在: Return(0)
 - 新模式无可用充电桩: Return(0)

契约7: Query_Car_State(car_id)

- **操作:** Query_Car_State(car_id: String)
- **交叉引用:** UC-03 查看排队状态
- **前置条件:**
 - i. 用户已登录
 - ii. car_id 对应的充电请求存在
- **后置条件:**
 - i. 返回 car_Number_before_position (前面排队车辆数)
 - ii. 返回 car_state (排队中/等待中/充电中/已完成)
 - iii. 返回 queue_Num (所在队列编号)
 - iv. 返回 request_time (请求时间)
- **异常处理:**
 - 请求不存在: Return 空

契约8: Start_Charging(car_id, ChargePileNum)

- **操作:** Start_Charging(car_id: String, ChargePileNum: String)
- **交叉引用:** UC-08 确认开始充电
- **前置条件:**
 - i. 充电请求存在且状态为"等待中"
 - ii. 车辆在等待区排到首位
 - iii. 充电桩 ChargePileNum 状态为可用
- **后置条件:**
 - i. 创建 ChargingSession 对象
 - ii. 请求状态更新为"充电中"
 - iii. 充电桩状态更新为"充电中"

iv. 记录开始充电时间

- **异常处理:**

- 充电桩不可用: Return(0)
- 操作超时: 取消充电并扣除基本服务费

契约9: Query_Charging_State(car_id)

- **操作:** Query_Charging_State(car_id: String)

- **交叉引用:** UC-10 查看充电进度

- **前置条件:**

- i. car_id 对应的充电会话存在且状态为"充电中"

- **后置条件:**

- i. 返回详单信息: 当前电量、已充电量、充电功率、预计剩余时间、当前时段电价、已产生费用等

- **异常处理:**

- 会话不存在: Return 空

契约10: End_Charging(car_id, ChargingPileNum)

- **操作:** End_Charging(car_id: String, ChargingPileNum: String)

- **交叉引用:** UC-11 结束充电

- **前置条件:**

- i. 充电会话存在且状态为"充电中"
- ii. 到达目标电量或用户主动终止
- iii. ChargingPileNum 与当前充电桩匹配

- **后置条件:**

- i. 充电会话状态更新为"已完成"
- ii. 记录结束时间和实际充电量
- iii. 释放充电桩资源
- iv. 生成账单记录 (BillingRecord)
- v. 请求状态更新为"已完成"

- **异常处理:**

- ChargingPileNum 不匹配: Return(0)
- 会话不存在: Return(0)

契约11: Request_Bill(carId, date)

- **操作:** Request_Bill(carId: String, date: Date)

- **交叉引用:** UC-12 查看账单

- **前置条件:**

- i. 用户已登录
- ii. 日期有效

- **后置条件:**

- i. 查询指定日期该用户的所有账单记录
- ii. 返回每条账单的概览信息 (11个字段)

- **异常处理:**

- 无账单记录: Return null
- 日期无效: Return(0)

契约12: Request_DetailedList(Bill_Id)

- **操作:** Request_DetailedList(Bill_Id: String)
- **交叉引用:** UC-12a 查看详细单
- **前置条件:**
 - i. Bill_Id 对应的账单存在
- **后置条件:**
 - i. 返回账单基础信息
 - ii. 返回各时段分段明细
- **异常处理:**
 - Bill_Id 不存在: Return(0)

3.4.2 管理员相关操作契约

契约13: powerOn(pile_Id)

- **操作:** powerOn(pile_Id: String)
- **交叉引用:** UC-17 控制充电桩可用性
- **前置条件:**
 - i. 管理员已登录
 - ii. 充电桩存在且处于关闭状态
- **后置条件:**
 - i. 充电桩电源开启
 - ii. 充电桩状态更新为"可用"
 - iii. 记录操作日志
- **异常处理:**
 - 充电桩不存在: Return(0)
 - 充电桩已在运行: Return(0)

契约14: setParameters(pile_Id, tariffConfig, peakPrice, normalPrice, valleyPrice)

- **操作:** setParameters(pile_Id: String, tariffConfig, peakPrice: Double, normalPrice: Double, valleyPrice: Double)
- **交叉引用:** UC-18 修改充电桩参数
- **前置条件:**
 - i. 管理员已登录
 - ii. 充电桩存在
 - iii. 电价数据有效 (>0)
- **后置条件:**
 - i. 充电桩的计费规则配置更新
 - ii. 三个时段电价数据更新
 - iii. 记录参数变更日志
- **异常处理:**
 - 电价数据无效: Return(0)
 - 充电桩不存在: Return(0)

契约15: Start_ChargingPile(pile_Id)

- **操作:** Start_ChargingPile(pile_Id: String)
- **交叉引用:** UC-23 运行充电桩

- **前置条件：**
 - i. 管理员已登录
 - ii. 充电桩状态为"可用"
 - iii. 充电桩已完成参数配置
- **后置条件：**
 - i. 充电桩状态更新为"运行中"
 - ii. 充电桩开始接受充电请求
 - iii. 记录启动日志
- **异常处理：**
 - 充电桩未就绪：Return(0)
 - 参数未配置：Return(0)

契约16：powerOff(pile_Id)

- **操作：**powerOff(pile_Id: String)
- **交叉引用：**UC-17 控制充电桩可用性
- **前置条件：**
 - i. 管理员已登录
 - ii. 充电桩存在
 - iii. 充电桩无正在充电的车辆
- **后置条件：**
 - i. 充电桩电源关闭
 - ii. 充电桩状态更新为"关闭"
 - iii. 记录操作日志
- **异常处理：**
 - 充电桩正在使用中：Return(0)
 - 充电桩不存在：Return(0)

契约17：Query_PileState(pile_Id)

- **操作：**Query_PileState(pile_Id: String)
- **交叉引用：**UC-14 监控充电站状态
- **前置条件：**
 - i. 管理员已登录
 - ii. 充电桩存在
- **后置条件：**
 - i. 返回 workingState（工作状态）
 - ii. 返回 TotalChargeNum（累计充电次数）
 - iii. 返回 TotalChargeTime（累计充电时长）
 - iv. 返回 TotalCapacity（累计充电容量）
- **异常处理：**
 - 充电桩不存在：Return(0)

契约18：Query_QueueState(queueList)

- **操作：**Query_QueueState(queueList: List)
- **交叉引用：**UC-15 查看队列详情
- **前置条件：**

- i. 管理员已登录
- ii. queuelist 非空
- 后置条件：
 - i. 遍历每个队列中的所有车辆
 - ii. 返回每辆车的 car_Id, car_Capacity, Request_Amount, waitTime
- 异常处理：
 - 队列为空：Return 空列表

3.4.3 故障调度相关操作契约

契约19：rescheduleByPriority(pile_Id)

- 操作：rescheduleByPriority(pile_Id: String)
- 交叉引用：UC-19 处理充电桩故障（优先级调度）
- 前置条件：
 - i. 充电桩发生故障
 - ii. 管理员选择调度策略为"优先级调度"
 - iii. 故障充电桩上有车辆
- 后置条件：
 - i. 受影响车辆按优先级排序
 - ii. 每辆车按优先级顺序分配最优可用充电桩
 - iii. 无可用车的车辆加入全局等待队列最前
 - iv. 故障桩状态更新为"故障关闭"
- 异常处理：
 - 所有充电桩均不可用：通知管理员人工介入

契约20：rescheduleByTimeOrder(pile_Id)

- 操作：rescheduleByTimeOrder(pile_Id: String)
- 交叉引用：UC-19 处理充电桩故障（时间顺序调度）
- 前置条件：
 - i. 充电桩发生故障
 - ii. 管理员选择调度策略为"时间顺序调度"
 - iii. 故障充电桩上有车辆
- 后置条件：
 - i. 受影响车辆按请求时间升序排列
 - ii. 每辆车按时间顺序依次分配可用充电桩
 - iii. 故障桩状态更新为"故障关闭"
- 异常处理：
 - 所有充电桩均不可用：通知管理员人工介入

契约21：recoverChargingFault(pile_Id)

- 操作：recoverChargingFault(pile_Id: String)
- 交叉引用：UC-19 处理充电桩故障（充电中故障恢复）
- 前置条件：
 - i. 充电桩在充电过程中发生故障
 - ii. 故障桩上有正在充电的车辆

- **后置条件：**
 - i. 充电中会话暂停，保存已充电量快照
 - ii. 充电中车辆优先调度至同类型可用充电桩
 - iii. 充电会话从已充电量继续，记录中断时长
 - iv. 排队区和等待区车辆按时间顺序重新调度
 - v. 故障桩状态更新为"故障关闭"
- **异常处理：**
 - 无同类型可用桩：会话加入故障恢复等待队列

契约22：rescheduleByShortestTotalTime(vehicles)（故障场景）

- **操作：**rescheduleByShortestTotalTime(vehicles: List)
- **交叉引用：**UC-19 处理充电桩故障（单次调度最短时长）
- **前置条件：**
 - i. 充电桩发生故障
 - ii. 管理员选择故障策略为"单次调度最短时长"
 - iii. 故障桩上有车辆
- **后置条件：**
 - i. 对每台受影响车辆，遍历所有兼容桩计算 $T_j = w_j + c_j$
 - ii. 每台车辆独立选择 $\min(T_j)$ 的充电桩
 - iii. 车辆分配至目标充电桩排队区
 - iv. 故障桩状态更新为"故障关闭"
- **异常处理：**
 - 车辆无兼容桩：标记该车辆，通知管理员人工介入

契约23：batchAssignByShortestTotalTime(vehicles, piles)（故障场景）

- **操作：**batchAssignByShortestTotalTime(vehicles: List, piles: List)
- **交叉引用：**UC-19 处理充电桩故障（批量调度最短时长）
- **前置条件：**
 - i. 充电桩发生故障
 - ii. 管理员选择故障策略为"批量调度最短时长"
 - iii. 故障桩上有车辆
 - iv. piles 中至少有一个可用充电桩
- **后置条件：**
 - i. 构建 $N \times M$ 成本矩阵
 - ii. 执行匈牙利算法求最小权完美匹配
 - iii. 按最优分配方案批量分配车辆
 - iv. 故障桩状态更新为"故障关闭"
 - v. 记录批量调度总完成时间
- **异常处理：**
 - $N > M$ 时：分批处理
 - 车辆无任何兼容桩：标记为"待处理"

3.4.4 策略管理相关操作契约

契约24：initDispatchStrategy(algorithm, faultStrategy)

- **操作：**initDispatchStrategy(algorithm: String, faultStrategy: String)
- **交叉引用：**UC-42 设置启动默认策略
- **前置条件：**
 - i. 系统启动时，配置文件提供有效的策略标识符
- **后置条件：**
 - i. 创建 DispatchStrategy 配置对象
 - ii. 正常分配算法加载为 algorithm 对应的策略
 - iii. 故障处理策略加载为 faultStrategy 对应的策略
 - iv. 后续所有调度操作按当前激活策略执行
- **异常处理：**
 - 无效策略标识符：使用默认值（SHORTEST_TOTAL_TIME 和 TIME_ORDER）

契约25：switchDispatchStrategy(strategyType)

- **操作：**switchDispatchStrategy(strategyType: String)
- **交叉引用：**UC-43 运行时切换策略
- **前置条件：**
 - i. 管理员已登录
 - ii. 系统运行中
 - iii. strategyType 有效
- **后置条件：**
 - i. 系统当前激活的分配策略切换为 strategyType
 - ii. 后续所有正常分配操作使用新策略
 - iii. 记录策略切换日志
- **异常处理：**
 - 策略标识符无效：Return(0)，策略不变

契约26：switchFaultStrategy(faultType)

- **操作：**switchFaultStrategy(faultType: String)
- **交叉引用：**UC-43 运行时切换策略
- **前置条件：**
 - i. 管理员已登录
 - ii. 系统运行中
 - iii. faultType 有效
- **后置条件：**
 - i. 系统当前激活的故障处理策略切换为 faultType
 - ii. 后续所有故障处理操作使用新策略
 - iii. 记录策略切换日志
- **异常处理：**
 - 策略标识符无效：Return(0)，策略不变

3.5 指令对应表

用例	指令	参数	返回
注册			

用例	指令	参数	返回
1、创建新账号	createNewAccount(car_Id, userName, car_Capacity)	car_Id: 车牌号, userName: 用户名, car_Capacity: 电池容量	Return(0/1)
2、验证数据	set_pwd(*****)	password: 密码	Return(0/1)
登录			
1、登录系统	login(car_Id, password)	car_Id: 车牌号, password: 密码	Return(0/1)
充电申请			
1、 提交充电申请	E_chargingRequest(car_Id, Request_Amount, Request_Mode)	car_Id: 车牌号, Request_Amount: 请求电量, Request_Mode: 充电模式	Return(car_position, car_state, queue_Num, request_time)
2、修改充电量	Modify_Amount(car_Id, Amount)	car_Id: 车牌号, Amount: 新电量	Return(0/1)
3、 修改充电模式	Modify_Mode(car_Id, Mode)	car_Id: 车牌号, Mode: 新模式	Return(0/1)
4、 查看队列状态	Query_Car_State(car_id)	car_id: 车牌号	Return(car_Number_before_position, car_state, queue_Num, request_time)
5、开始充电	Start_Charging(car_id, ChargePileNum)	car_id: 车牌号, ChargePileNum: 充电桩编号	Return(0/1)
6、 查看充电状态	Query_Charging_State(car_id)	car_id: 车牌号	Return(详单信息)
7、结束充电	End_Charging(car_id, ChargingPileNum)	car_id: 车牌号, ChargingPileNum: 充电桩编号	Return(0/1)
查看账单			
1、 查看账单申请	Request_Bill(carId, date)	carId: 车牌号, date: 日期	Return(11个字段)
查看详细单 (子)			
1、 查看详细单申请	Request_DetailedList(Bill_Id)	Bill_Id: 账单编号	Return(基础信息+分段明细)

用例	指令	参数	返回
运行充电桩			
1、启动充电桩	powerOn(pile_Id)	pile_Id: 充电桩编号	Return(0/1)
2、设置参数	setParameters(pile_Id, 计费规则, 峰时电价, 平时电价, 谷时电价)	pile_Id: 充电桩编号, 电价数据	Return(0/1)
3、运行充电桩	Start_ChargingPile(pile_Id)	pile_Id: 充电桩编号	Return(0/1)
4、关闭充电桩	powerOff(pile_Id)	pile_Id: 充电桩编号	Return(0/1)
查看充电桩状态			
1、 查看充电桩状态	Query_PileState(pile_Id)	pile_Id: 充电桩编号	Return(workingState, TotalChargeNum, TotalChargeTime, TotalCapacity)
查看队列状态			
1、 查看队列状态	Query_QueueState(queueList)	queueList: 队列列表	Return(car_Id, car_Capacity, Request_Amount, waitTime)
管理调度策略			
1、 启动默认策略	initDispatchStrategy(algorithm, faultStrategy)	algorithm: 分配策略标识符, faultStrategy: 故障策略标识符	策略就绪
2、 切换分配策略	switchDispatchStrategy(strategyType)	strategyType: 分配策略标识符	Return(0/1)
3、 切换故障策略	switchFaultStrategy(faultType)	faultType: 故障策略标识符	Return(0/1)

四、系统事件人员分配

4.1 专业分工方向

组员	角色	专业方向
杜昊阳	组长	系统架构设计与核心调度策略
赫金科	组员	领域数据建模与静态结构设计
康忆文	组员	用户服务流程与充电业务交互

组员	角色	专业方向
陆昱衡	组员	计费账单与管理功能设计
米梓润	组员	调度优化策略与文档规范

4.2 用户端事件分配

用例	指令	负责人
注册	createNewAccount(car_Id, userName, car_Capacity)	康忆文
注册	set_pwd(*****)	康忆文
登录	login(car_Id, password)	康忆文
充电申请	E_chargingRequest(car_Id, Request_Amount, Request_Mode)	康忆文
充电申请	Modify_Amount(car_Id, Amount)	康忆文
充电申请	Modify_Mode(car_Id, Mode)	康忆文
充电申请	Query_Car_State(car_id)	米梓润
充电申请	Start_Charging(car_id, ChargePileNum)	陆昱衡
充电申请	Query_Charging_State(car_id)	陆昱衡
充电申请	End_Charging(car_id, ChargingPileNum)	陆昱衡
查看账单	Request_Bill(carId, date)	陆昱衡
查看详单	Request_DetailedList(Bill_Id)	陆昱衡

4.3 管理员端事件分配

用例	指令	负责人
运行充电桩	powerOn(pile_Id)	赫金科
运行充电桩	setParameters(计费规则，三个时段的电价数据等)	赫金科
运行充电桩	Start_ChargingPile(pile_Id)	赫金科
运行充电桩	powerOff(pile_Id)	赫金科
查看充电桩状态	Query_PileState(pile_Id)	米梓润
查看队列状态	Query_QueueState(queueList)	米梓润

4.4 故障调度事件分配

调度策略	核心逻辑	负责人
优先级调度（SSD-F1）	按区域+会员+等待时间排序，依次分配最优桩	杜昊阳

调度策略	核心逻辑	负责人
时间顺序调度（SSD-F2）	按请求时间先来后到排序，依次分配可用桩	杜昊阳
充电中故障恢复（SSD-F3）	暂停会话+保存快照+优先恢复充电中车辆	杜昊阳
单次调度最短时长（SSD-F4）	遍历兼容桩，每台车独立选T最短（贪心）	米梓润
批量调度最短时长（SSD-F5）	成本矩阵+匈牙利算法求全局最优分配	米梓润

4.5 静态结构设计分配

任务	内容	负责人
领域模型类图修正	关系调整（去除组合关联面向关联）、 新增DispatchStrategy/TariffConfig/DetailedBill类	赫金科
用例级静态结构类图	6组用例类图（注册登录/充电申请/账单详单/运行充电桩/充电桩队列状态/调度策略）+ 类说明表	赫金科
活动图修改	注册登录分离入口 + 五种故障策略分支 + 账单详单两级查询	赫金科

4.6 其他任务分配

任务	内容	负责人
系统架构选择及说明	架构风格选择与对比、分层架构理由、部署拓扑图、废弃方案分析	杜昊阳
调度策略可切换机制设计	启动参数配置文件 + 运行时切换的SSD和操作契约	米梓润
文档整合与规范性检查	汇总组员内容，检查格式一致性，确保符合作业模板要求	杜昊阳
工作量统计	填写工作量统计表	杜昊阳

五、工作量统计

表：作业工作内容及工作量统计

任务类别	具体任务	杜昊阳	赫金科	康忆文	陆昱衡	米梓润
动态结构	注册+登录SSD及操作契约			●		
动态结构	充电申请SSD及操作契约（7条指令）		●	●	●	●
动态结构	查看账单+详单SSD及操作契约				●	
动态结构	运行充电桩SSD及操作契约（4条指令）		●			
动态结构	查看充电桩状态+队列状态SSD及操作契约			●		●
动态结构	优先级调度交互图（SSD-F1）	●				

任务类别	具体任务	杜昊阳	赫金科	康忆文	陆昱衡	米梓润
动态结构	时间顺序调度交互图（SSD-F2）	●				
动态结构	充电中故障恢复交互图（SSD-F3）	●				
动态结构	单次调度最短时长交互图（SSD-F4）					●
动态结构	批量调度最短时长交互图（SSD-F5）					●
动态结构	调度策略切换交互图				●	●
静态结构	领域模型类图修正		●			
静态结构	用例级静态结构类图（6组）		●			
静态结构	活动图修改		●			
架构	系统架构选择及说明	●				
架构	调度策略可切换机制					●
文档	整合、规范性检查、人员分配表	●				
文档	工作量统计	●				